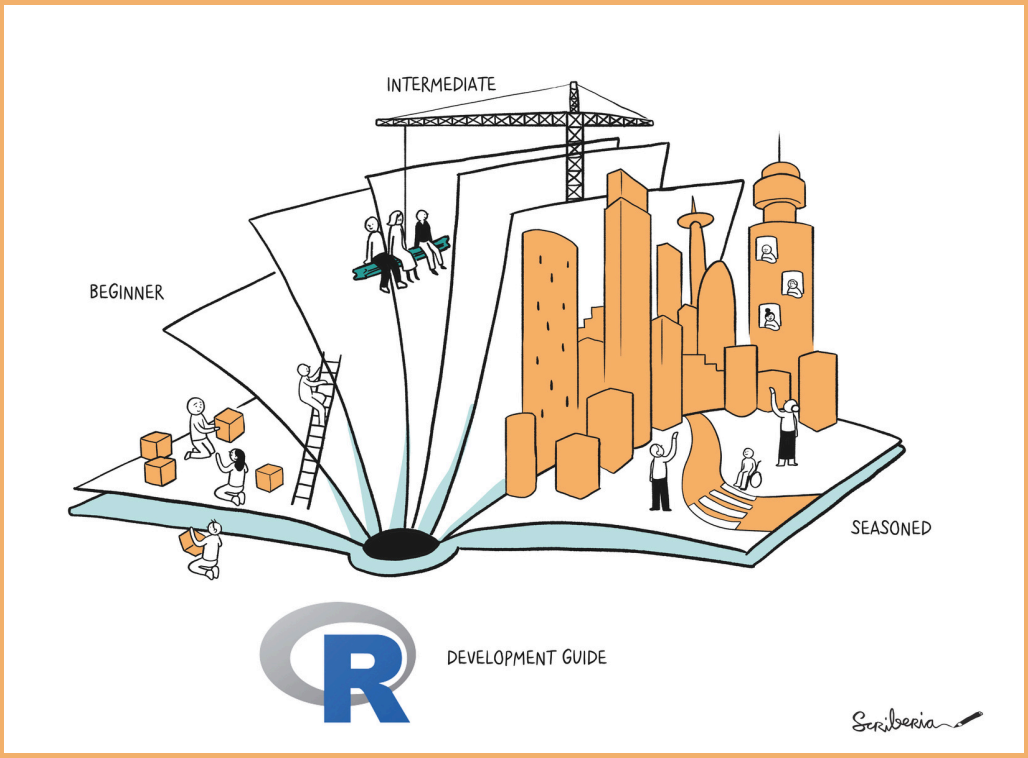
R Development Guide



R Development Guide. This illustration is created by Scriberia with The Turing Way community, used under a CC-BY 4.0 licence. DOI: https://zenodo.org/records/13882307

collaboratively authored by the

R Contribution Working Group

R Development Guide

R Contribution Working Group

2026-11-13

Table of contents

Acknowledgement 6

1 Introduction 8
- 1.1 Overview of different ways of contributing to the base R Project . . . 8
- 1.2 Quick start to the guide . . . 8
- 1.3 How to contribute to this guide itself? . . . 9

2 R Patched and Development Versions 11
- 2.1 The R source code . . . 11
- 2.2 Prerequisites . . . 12
  - 2.2.1 Ubuntu . . . 12
  - 2.2.2 Fedora . . . 12
- 2.3 Building R . . . 12
  - 2.3.1 Linux . . . 13
  - 2.3.2 Windows . . . 14
  - 2.3.3 macOS . . . 16
- 2.4 See also . . . 16

3 Issue Tracking 17
- 3.1 How to contribute to issue tracking? . . . 17
- 3.2 How to get a Bugzilla account? . . . 17
  - 3.2.1 Navigating Bugzilla . . . 17
- 3.3 Searching for Bugs to Contribute to . . . 19
  - 3.3.1 Detailed Bug Information . . . 21
  - 3.3.2 Search By People . . . 21
  - 3.3.3 Search By Change History . . . 22

4 Reporting Bugs 23
- 4.1 What is a bug in R? . . . 23
- 4.2 What condition might not be a bug? . . . 24
- 4.3 Checking if a bug is already reported . . . 24
- 4.4 What are some places where you may find a bug? . . . 25
- 4.5 How to report a bug? . . . 25
  - 4.5.1 Bug in the R-Core supported packages, their documentations, and/ or in the R language . . . 25
  - 4.5.2 Bug in the non R-Core supported packages and/or their documentations 25

4.5.3 Bug in a documentation website managed by the R Core Team 26
4.6 Good practices in reporting bugs / Expectations of a good bug report 26
4.7 Disagreement with a resolution on the bug tracker 27
4.8 Examples of Bug reports submitted on Bugzilla or R-devel mailing list 27
4.9 See also 28

5 Submitting Feature Requests **29**
5.1 What is a feature request? 29
5.2 How to submit a feature request? 29
5.3 Examples of feature requests submitted on Bugzilla 30

6 Reviewing Bugs **31**
6.1 How you can help to review bug reports? 31
6.1.1 Preparing to review bug reports 31
6.2 Classifying bug reports 31
6.3 How to find a bug report or an issue to review? 32
6.4 Example of a bug review submitted on Bugzilla 32
6.5 See also 32

7 Finding the Source **33**
7.1 Finding R source code 33
7.2 Finding C source code 34
7.3 See also 35

8 Lifecycle of a Patch **36**
8.1 Introduction 36
8.2 When do you submit a patch? 36
8.3 What tools are required to submit a patch? 36
8.4 How to prepare a patch? 37
8.4.1 Using a git mirror 37
8.5 Making good patches 41
8.6 Submitting your patch for review 42
8.6.1 Patch in response to a pre-existing issue or bug report 42
8.6.2 Patch in response to an unreported issue or bug report 43
8.7 Getting your patch reviewed 43
8.7.1 How to review a patch? 43
8.8 Leaving a patch review on Bugzilla 44
8.9 Dismissing review from another core developer 44
8.10 Acceptance or rejection of your patch 45
8.11 Examples of patch reports on Bugzilla 45
8.12 Examples of reviewing a patch 45
8.13 See also 45

9 Documenting **46**
9.1 Helping with documentation . . . 46
9.2 Reporting documentation bugs . . . 47
9.2.1 Example documentation bugs . . . 47
9.3 Guidelines for writing R help files . . . 47
9.4 Introduction to `.Rd` files . . . 48
9.5 R manuals . . . 51
9.6 Proofreading . . . 52
9.7 Helping with the R Development Guide . . . 52
9.8 See also . . . 52

10 Message Translations **53**
10.1 How translations work . . . 53
10.1.1 `.pot` files . . . 53
10.1.2 `.po` files . . . 54
10.1.3 `.mo` files . . . 56
10.2 How to contribute new translations . . . 56
10.3 Bulk Translations . . . 60
10.4 Current status of translations in R . . . 61
10.5 Helpful references . . . 61

11 Testing Pre-release R Versions **62**
11.1 Where to test? . . . 62
11.1.1 Virtual machine . . . 62
11.2 What can you test? . . . 62
11.3 Writing tests for R . . . 63
11.4 Benchmarks . . . 63

12 Where to Get Help **64**
12.1 Slack . . . 64
12.2 Mailing lists . . . 65
12.3 File a bug . . . 65

13 News and Announcements **66**
13.1 Blogs . . . 66
13.2 Conferences . . . 66
13.3 Journal . . . 66
13.4 Mailing lists . . . 66
13.5 Twitter . . . 67

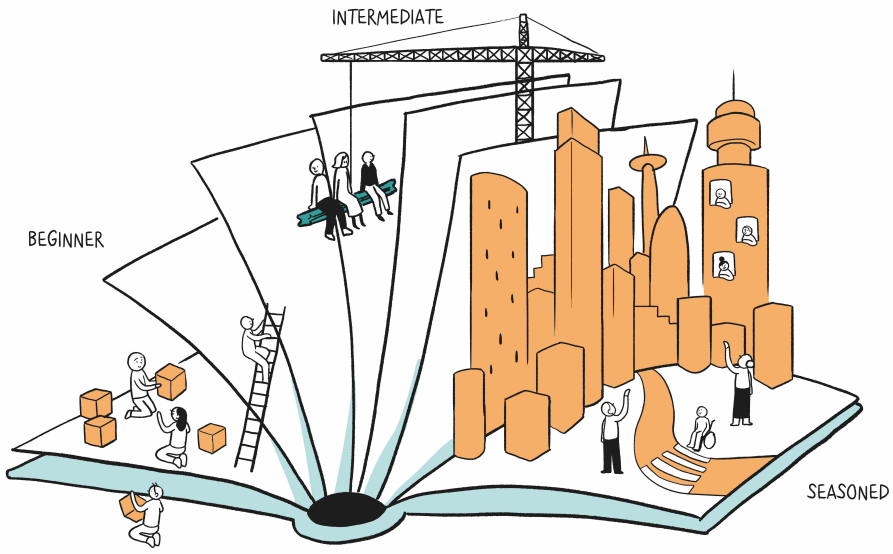
14 Developer Tools **68**
14.1 Subversion (svn) client . . . 68
14.2 Globally search for a regular expression and print matching lines (grep) . . . 68

14.3 Git 68
14.4 GitHub 69

15 Additional resources **70**
15.1 R Contributor site 70
15.2 R developer page 70
15.3 R manuals 70

16 R Core Developers **71**

Acknowledgement



This guide draws on documentation and articles written by the R Core Team. The first version of the guide was heavily influenced by the Python Developer's Guide.

Initial chapters of the guide were developed by Saranjeet Kaur Bhogal, in a project funded by the R Foundation, mentored by Heather Turner and Michael Lawrence. This initial version was upgraded in a Google Season of Docs 2022 project with Saranjeet Kaur Bhogal and Lluís Revilla Sancho working as technical writers managed by Nicolas Bennett and overseen by a Steering Committee including representatives from R Core and the wider R community.

This guide has benefited and continues to benefit from varied contributions by several contributors.



This project is licensed under a Creative Commons Attribution 4.0 International (CC BY 4.0). Some pages may contain materials that are subject to copyright, in which case you will see the copyright notice.

1 Introduction

This guide is a comprehensive resource for contributing to base R[1] – for both new and experienced contributors. It is maintained by the R Contribution Working Group. We welcome your contributions to base R!

1.1 Overview of different ways of contributing to the base R Project

Contributions to base R are possible in a number of different ways. Some of them are listed below:

1. Contributing to bug fixing: Refer to Issue Tracking and Reviewing Bugs.
2. Contributing to translations: Refer to Message Translations.
3. Testing R before release: Refer to Testing Pre-release R Versions.
4. Contributing to documentation, including this guide: Refer to Documenting R and Contributing to this guide.

1.2 Quick start to the guide

The guide is intended as a comprehensive resource for contributing to base R. The following chapter outline provides an overview with links to sections for getting started with contributing.

- The Introduction chapter provides an overview of different ways of contributing to the R project, a quick start to the guide, and a section on how to contribute to the guide itself.
- The R Patched and Development Versions chapter covers instructions on how to install R from source or from binaries and also discusses the tools required to build R.
- The Issue Tracking chapter covers how contributors can support issue tracking in R, how to get an account on R's Bugzilla, how to navigate Bugzilla and search for bugs to contribute to.
- The Reporting Bugs chapter discusses what condition may or may not be a bug and how to report bugs if you find them.

[1]The set of packages in the base R distribution that are maintained by the R Core Team.

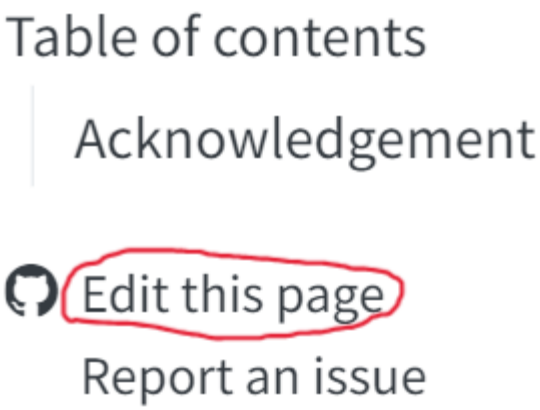
- The Submitting Feature Requests chapter discusses how to submit feature requests and shares some examples submitted to Bugzilla.
- The Reviewing Bugs chapter discusses how you can make a contribution to reviewing bug reports.
- The Finding the Source chapter provides an overview of the R codebase and helps with finding source code of base functions written in R and/or C.
- The Lifecycle of a Patch chapter discusses how to create a patch to propose a bug fix.
- The Documenting chapter describes the format and style guide for help files in R, how to report and review issues in the existing documentation, and how to propose changes.
- The Message Translations chapter covers internationalization in R, i.e., the display of messages in languages other than English.
- The Testing Pre-release R Versions chapter discusses how you can help with testing of pre-release versions of R.
- For more information on how to engage with the community and ask for help, refer to the Where to Get Help chapter.
- To keep up with the developments in R refer to the resources available in the News and Announcements chapter.
- Tools that may be useful for R developers are available in the Developer Tools chapter.
- Additional resources for contributing to R are available in the Additional Resources chapter.
- The R Core Developers chapter contains a list of the former and current members of the R Core Team who have write access to the R source.

1.3 How to contribute to this guide itself?

All contributors should start by reading the contributing page for this project.

Maintainers and contributors are also requested to follow this guide's code of conduct.

This guide is built using Quarto which makes editing it easier, provided you have a GitHub account (sign-up at github.com). After you log-in to GitHub, click on the 'Edit this page' (available in the left side column) link highlighted with a red ellipse in the image below. This will take you to an editable version of the source R Markdown file that generated the page you are on:



Use the issue tracker to raise an issue about the guide's content or to make a feature request.

2 R Patched and Development Versions

These instructions cover how to install R from source or from binaries. Contributors will typically need to work with the patched or development versions of R. This chapter describes where the source code for these versions can be found and how to install these versions from the source or the binary builds (where available). The tools required to build R and R packages are also discussed. For the most up to date and complete instructions you can check the R installation and administration manual .

2.1 The R source code

R uses svn as a version control tool hosted at https://svn.r-project.org/R/ and uses a 'major.minor.patchlevel' version numbering scheme[1].

There are three releases of R available to install:

- The latest official release (`r-release`), either major version x.0.0 or minor version x.y.0, for example: 3.0.0 or 3.2.0
- The patched release (`r-patched`), for example 3.0.1 or 3.2.1 and
- The development release (`r-devel`) : continually developed version moving from r-release to next major/minor version (x + 1).0.0 or x.(y + 1).0 a few weeks before release (at the start of the "GRAND FEATURE FREEZE").

The source code of released versions of R can be found at R/tags, the patched versions are at R/branch.

The `r-devel` at R/trunk is the next minor or eventual major release development version of R. Bug fixes and new features are introduced in `r-devel` first. If the change meets the development guidelines R Core will also make the change in `r-patched`.

[1] Also known as semantic versioning

2.2 Prerequisites

To install from the source code you will need the source code and the dependencies of R.

If you need to install svn you can use your distribution's package manager to install it.

2.2.1 Ubuntu

In Ubuntu you can use this command to find all the dependencies of R:

```
apt-rdepends --build-depends --follow=DEPENDS r-base-dev | grep " B" | sed -e "s/  Build-Dep
```

It might require installation of apt-rdepends which can be done from default repositories via `sudo apt-get install apt-rdepends`.

To install all the R dependencies you can use:

```
sudo apt-get build-dep r-base-dev
```

2.2.2 Fedora

In Fedora you can use this command to find all the dependencies of R:

```
dnf rq -q --repo=fedora-source --requires R
```

You will also need the rsync package to download the recommended packages.

To install them you can use:

```
dnf install 'dnf-command(builddep)'
dnf install rsync
dnf builddep R
```

2.3 Building R

It is recommended to build R in a different path than the source. For this reason we have a `TOP_SRCDIR` variable where the source code goes and the variable `BUILDDIR` where the built R version will go.

2.3.1 Linux

Here are the basic steps intended as a checklist. For complete instructions please see the section in R-admin.

0. Install Subversion if not already available. On Debian/Ubuntu:

   ```
   sudo apt install subversion
   ```

   On Fedora/RHEL:

   ```
   sudo dnf install subversion
   ```

1. Retrieve R source code via into `TOP_SRCDIR`, note that we retrieve the `r-devel` source code:

   ```
   export TOP_SRCDIR="$HOME/Downloads/R"
   svn checkout https://svn.r-project.org/R/trunk/ "$TOP_SRCDIR"
   ```

2. Download the latest recommended packages[2]:

   ```
   "$TOP_SRCDIR/tools/rsync-recommended"
   ```

3. Create the build directory in the `BUILDDIR`:

   ```
   export BUILDDIR="$HOME/bin/R"
   mkdir -p "$BUILDDIR"
   cd "$BUILDDIR"
   ```

4. Configure the R installation (with `--enable-R-shlib` so that RStudio IDE can use it):

   ```
   "$TOP_SRCDIR/configure" --enable-R-shlib
   ```

5. Build R :

   ```
   make
   ```

6. Check that R works as expected:

   ```
   make check
   ```

   There are other checks you can run:

   ```
   make check-devel
   make check-recommended
   ```

If we don't want to build R in a different directory than the source code we can simply use:

[2]Recommended packages are not in the subversion repository.

```
cd "$TOP_SRCDIR"
svn update
tools/rsync-recommended
"$TOP_SRCDIR/configure"  --enable-R-shlib
make
make check
```

Once you successfully built R from source you can modify the R source code to fix an issue: Prepare a patch (See this guide) and after checking that R works as intended (`make check-devel`) submit the patch for consideration by R Core. (See the lifecycle of a patch chapter).

To use the `r-devel` version in RStudio, you can do the following:

```
export RSTUDIO_WHICH_R="$BUILDDIR/bin/R"
cd "$TOP_SRCDIR"
rstudio
```

2.3.2 Windows

2.3.2.1 Binaries

The binary builds of R for Windows can be downloaded and installed from here. Along with the link to the latest stable release, this page also contains links to the binary builds of `r-patched` and `r-devel`.

1. Click on the download links to download an executable installer.
2. Select the language while installing, read the GNU general public license information, and select the destination location to start the installation. You will be prompted to select components at this stage: `User installation`, `64-bit User installation`, or `Custom installation`. The default option may be chosen for the questions from this step onwards to complete the installation.

Daily binaries for `r-devel` are made available for download and installation.

2.3.2.2 From source

Before installing R from source, some additional programs are needed, as per the latest documentation:

1. Rtools is the suggested toolchain bundle for building R base and R packages containing compiled code on Windows. The latest version of Rtools can be installed using the Rtools installer rtools45-XXXX-XXX.exe.

2. A LaTeX compiler is needed to install and build R, check packages and build manuals. On CRAN, MiKTeX is used, which can be downloaded from https://miktex.org. Once installed open MiKTeX via the Windows start menu. It might ask to check for updates and more importantly, to make it available in PATH. You can accept both.

3. Open the Rtools45 terminal to update and install subversion:

```
pacman -Syuu
pacman -Sy wget subversion
```

4. Retrieve the latest source code via subversion:

```
export TOP_SRCDIR="$HOME/Downloads/R"
svn checkout https://svn.r-project.org/R/trunk/ "$TOP_SRCDIR"
```

   If you already have the repository available you can update as:

```
cd $TOP_SRCDIR
svn update
```

   You can also use a SVN client such as TortoiseSVN (https://tortoisesvn.net/, command line tool, and Windows Explorer integration) or SlikSVN (https://sliksvn.com/download/, just the command line tool) so that it can be also found by other tools.

5. Download the latest tcl/tk and unzip it in `$TOP_SRCDIR`:

```
cd "$TOP_SRCDIR"
wget -np -nd -r -l1 -A 'tcltk-*.zip' https://cran.r-project.org/bin/windows/Rtools/rtool
unzip "tcltk-*.zip"
```

6. Add gcc, MiKTeX and tar to the PATH and set one tar option:

```
export PATH="/x86_64-w64-mingw32.static.posix/bin:$PATH"
export PATH="/c/Program Files/MiKTeX/miktex/bin/x64:$PATH"
export TAR="/usr/bin/tar"
export TAR_OPTIONS="--force-local"
```

   If MiKTeX was installed just for your user you might need to run:

```
export PATH="/c/Users/$USER/AppData/Local/Programs/MiKTeX/miktex/bin/x64:$PATH"
```

7. Check that all the programs can be found:

```
which make gcc pdflatex tar
```

   If there is any error you'll need to find where the program is installed and add the corresponding path.

8. Download the latest recommended packages[3]:

```
cd "$TOP_SRCDIR/src/gnuwin32/"
"$TOP_SRCDIR/tools/rsync-recommended"
```

9. Build R and the recommended packages:

```
make all recommended
```

   The recently compiled version of R will be at `$TOP_SRCDIR/bin/`. In RStudio you can select that folder and restart it to use the `r-devel` version.

10. Check that R works as expected:

```
make check
```

   There are other checks you can run for testing the successful installation of the recommended packages:

```
make check-devel
make check-recommended
```

2.3.3 macOS

This section will be added after the official installation instructions for macOS in the R installation and administration manual have been updated for R 4.4.0.

2.4 See also

1. CRAN official website
2. R installation and administration manual
3. R for macOS
4. Tools for R in macOS
5. Requirements for R in macOS
6. R for Windows FAQ
7. RTools toolchains for Windows
8. R FAQ

[3]Recommended packages are not in the subversion repository.

3 Issue Tracking

Base R uses Bugzilla for issue tracking and reporting bugs.

3.1 How to contribute to issue tracking?

Contributors can support issue tracking in R by

1. Reporting bugs or submitting feature requests (bug reports filed under the "Wishlist" component in Bugzilla),
2. Reviewing bug reports to help close reports where no fix is needed or to help narrow down the problem and identify what needs to be fixed, or
3. Proposing changes to code or documentation that will close an issue.

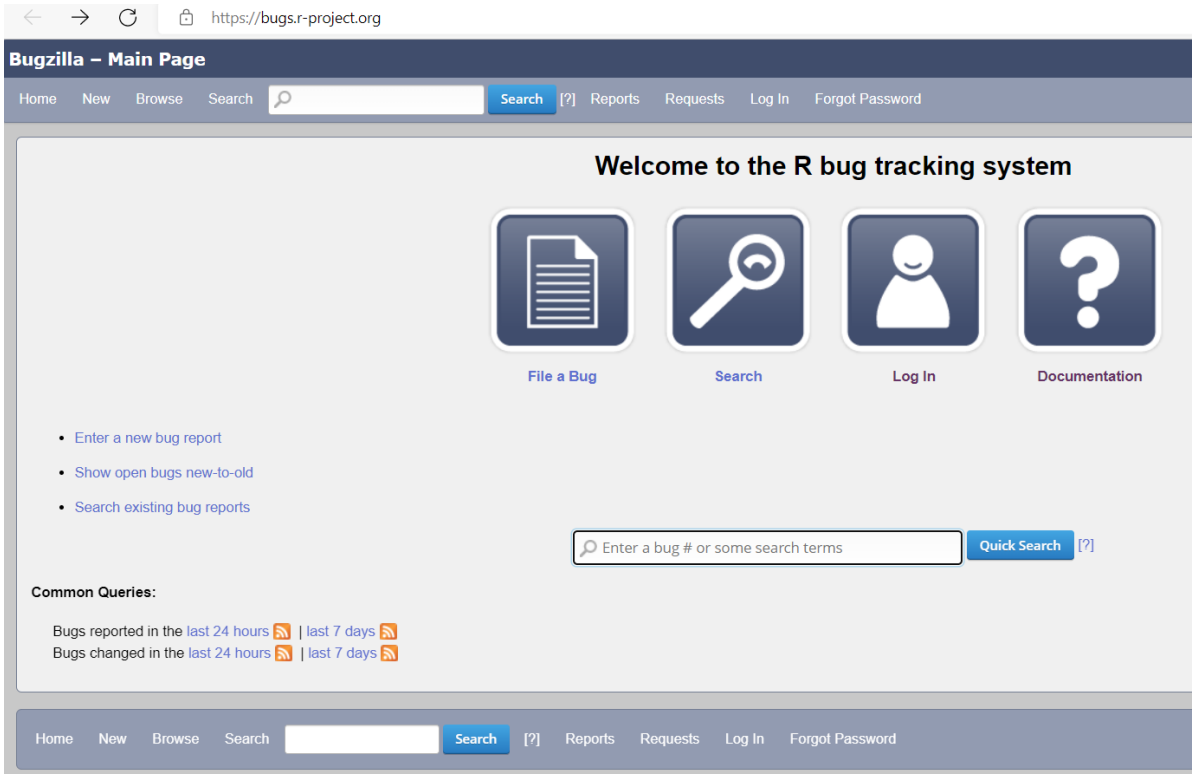
3.2 How to get a Bugzilla account?

Anyone can browse the bug reports on Bugzilla, but you will need an account to file a bug report, to comment on an existing bug report, or to submit a proposed fix in the form of patch file.

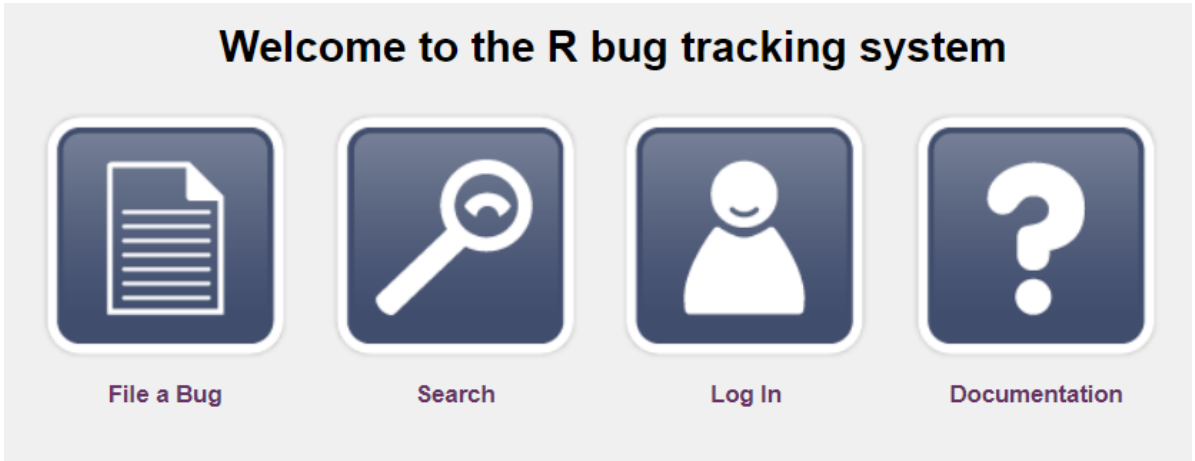
To get a Bugzilla account send an e-mail to `bug-report-request@r-project.org` from the address you want to use as your Bugzilla login. Briefly explain why you want a Bugzilla account and a volunteer will add you to R's Bugzilla members. After having successfully logged in to Bugzilla, you are good to go.

3.2.1 Navigating Bugzilla

An image of the existing home page of Bugzilla is shared below:



On the home page of Bugzilla, there are various buttons and links. There are four square buttons called as:



1. File a bug: You will have to log in to Bugzilla to file a bug using this button
2. Search: When you click this button you will get a page with 'Simple Search' and 'Advanced

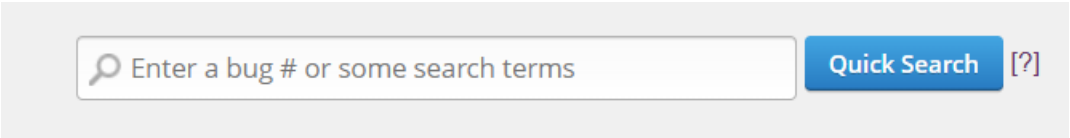
Search' options. Either of the search options could be used depending on what you are looking for.

3. Log In: Provide the appropriate email address and password here to log in to Bugzilla.
4. Documentation: Provides a user guide for R's Bugzilla.

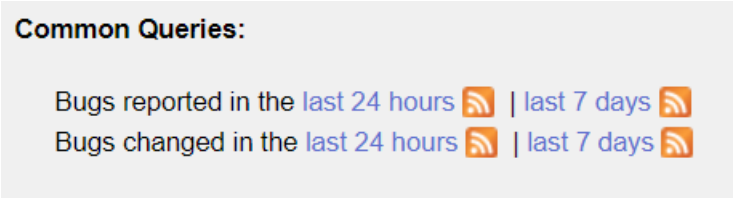
Several useful shortcuts are available from the landing page:

1. Enter a new bug report
2. Show open bugs new-to-old
3. Search existing bug reports

A quick search bar is available on the home page where you can enter a bug number to search or some search terms.

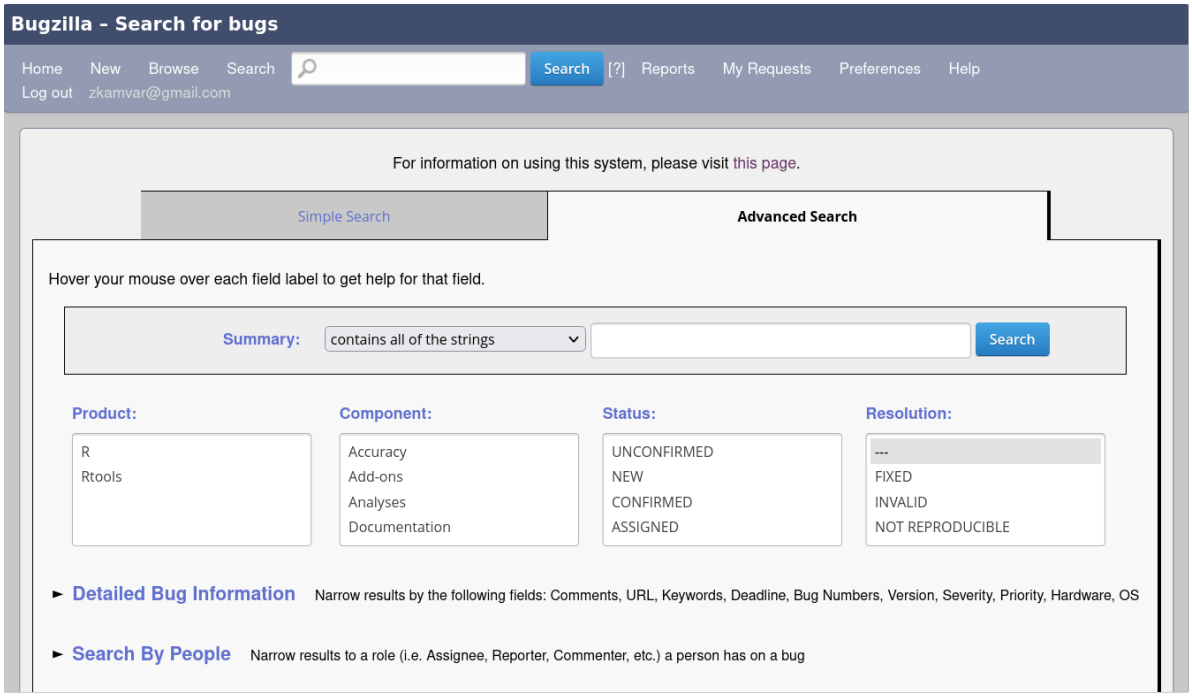


There is also a section for Common Queries on the home page which includes links to bugs reported and changed in the last 24 hours and last 7 days.



3.3 Searching for Bugs to Contribute to

When presented with a long list of bugs, it can often be a bit demotivating when it's not clear where to start. To help with this, you can use the **Advanced Search**.



This presents several fields that you can use to narrow down your search. You can find out what a particular field is by clicking on the header, which will take you to that section in the **Bug Fields Information Page**.

The default fields are:

- Summary: a short description of the bug
- Product: either R or RTools.
- Component: A specific component of the R toolchain such as Graphics, Documentation, and Models
- Status: The status of open or closed bugs. This will always default to showing open bugs (UNCONFIRMED, CONFIRMED, and IN_PROGRESS)
- Resolution: The resolution for closed bugs.

You can select multiple items for each field as well. Both **Product** and **Component** are what you navigate through when you go to **Browse Issues**.

Below that are further filters that can help you find bugs you might want to work on:

Detailed Bug Information: Narrow results by the following fields Comments, URL, Keywords, Deadline, Bug Numbers, Version, Severity, Priority, Hardware, OS Search By People
Narrow results to a role (i.e. Assignee, Reporter, Commenter, etc.) a person has on a bug Search By Change History

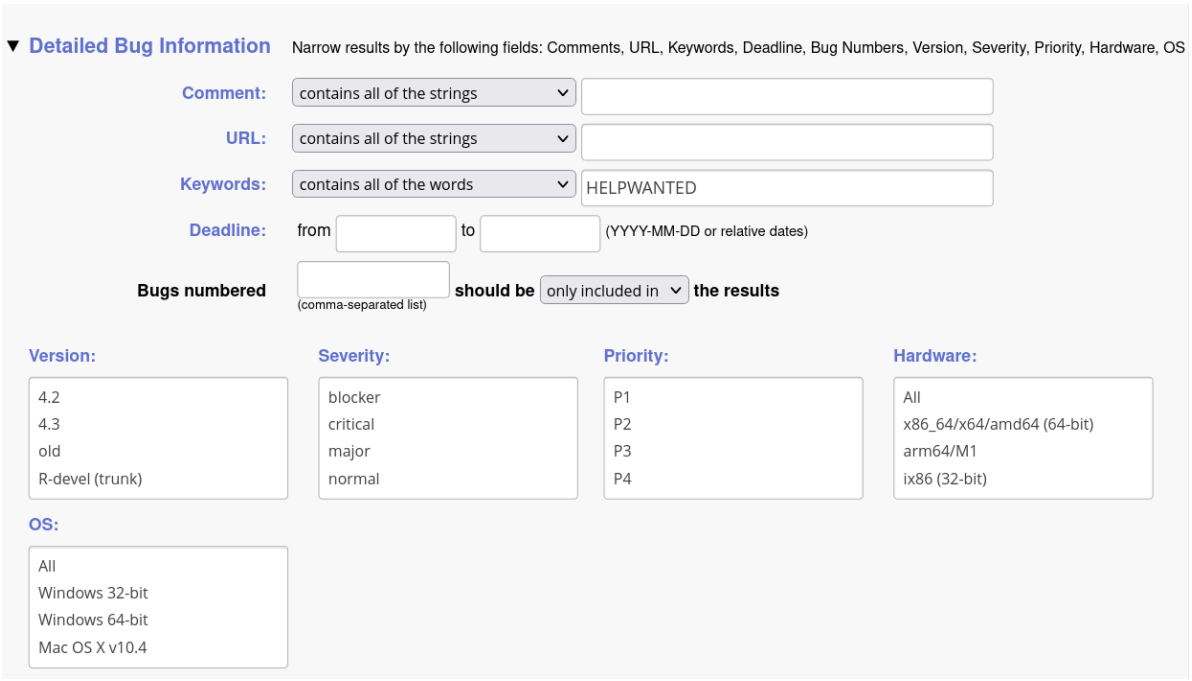
Narrow results to how fields have changed during a specific time period Custom Search Didn't find what you're looking for above? This area allows for ANDs, ORs, and other more complex searches.

These will be detailed in the subsections below

3.3.1 Detailed Bug Information

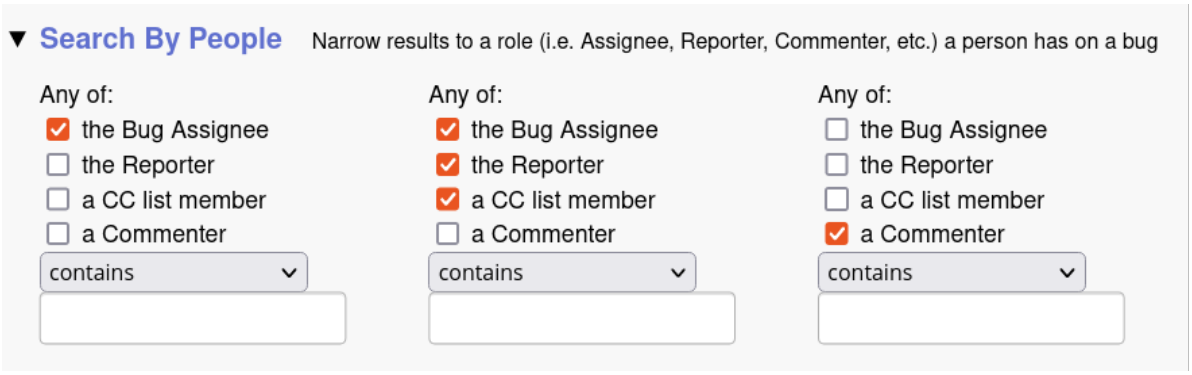
You can use this section to narrow down your search further to filter by things like OS type, R version, Keywords, and Priority.

It can be helpful, for example, to search for bugs labelled with `HELPWANTED`, which indicates bugs that may be suitable for anyone to work on regardless of expertise.



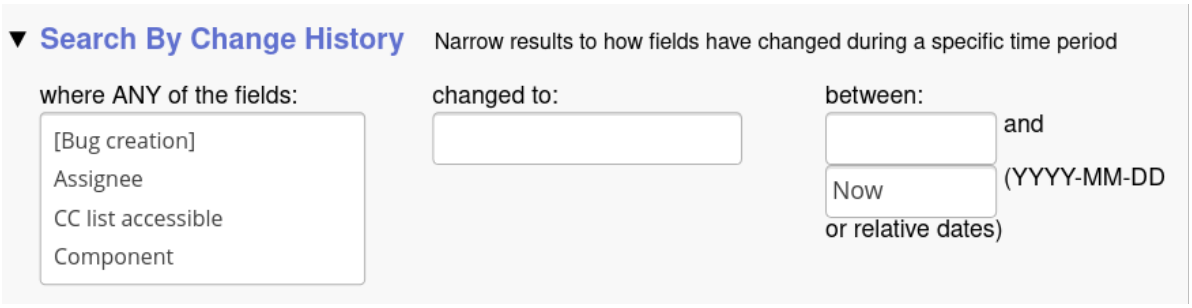
3.3.2 Search By People

You can use this to narrow down your search to only include results of a specific person or up to three people who have participated in any component of the bug's discussion.



3.3.3 Search By Change History

Use this to search for bugs where any of the status fields has been changed (which, depending on the change, could indicate that someone from R-Core has given some attention to the bug). This also provides a date range to narrow your search.



4 Reporting Bugs

4.1 What is a bug in R?

You may find a bug in R if:

1. The R session terminates unexpectedly, or there is a segmentation fault, it might be a bug in R, unless you have written your own call to compiled code or an internal function (via `.C` or `.Internal`). The error may look like this:

```
##  *** caught segfault ***
## address (nil), cause 'memory not mapped'
```

2. If the code does not do what the documentation says it should, then either the code or the documentation is wrong. Report either of which needs to be fixed.

Note: When you are in doubt that there is a bug: (which should be the case most of the time!)

1. Make sure whether the bug appears in a clean session of R. Many times, there are variables/commands/functions stored in the workspace which might cause issues. Hence, check if the issue happens in a clean session. To do so, launch R from the command line with the `--vanilla` option.
2. At times the code that is written is very complicated, has numerous package and file dependencies, has many function calls, etc.. In such scenarios it is quite common that the code throws an error and you are not able to solve it. You may tend to think that there is a bug that needs to be reported. Before doing so, try to produce a minimum working example of the code for the section where the error occurred. Add only those packages and files which are required by that section, and see if the error still appears. Using this approach shall solve most of the errors.
3. Install R-devel, which is the most recent version of R from svn / git or daily Windows build, and see if your bug still exists in R-devel (it may have been fixed very recently).

4. Search on the R-devel mailing list for messages with keywords related to your possible bug. If you find some related messages then read them to see if they clarify whether or not it is a bug. If you do not find any related messages, then please post a new message to R-devel. Your message should include (1) a brief description of the bug including current and expected behavior, (2) a minimal reproducible example.

4.2 What condition might not be a bug?

1. In case the code is doing something unexpected, it may not necessarily be a bug. Carefully review the documentation of the function being called, and check whether the behaviour being exhibited on calling this function is the same as it was designed to do.
2. Issues with *seemingly* identical numbers not being equal (especially floating point numbers) are usually not bugs.
3. If R is running slower than expected, then this also may not be a bug. Ask someone else to review your code in such a case.
4. If some function is working, but it is not defined in the best generalised way, then consult someone to look over the code. This may perhaps not be a bug; instead, it might be an alternative way of writing the function.

4.3 Checking if a bug is already reported

The first step before filing a bug report is to see whether the problem has already been reported. Checking if the bug is reported will:

1. Save time for you and the developers.
2. Help you see if the bug is already fixed for the next release.
3. Lead you to learn what needs to be done to fix it.
4. Determine if any additional information is needed.

The sections that follow discuss where to check whether a bug is already reported.

4.4 What are some places where you may find a bug?

You may find a bug in:

1. In the R-Core supported packages, their documentations, and/ or in the R language implementation.
2. In packages and/or their documentations which are not supported by the R-Core.

4.5 How to report a bug?

Once you confirm a bug exists, you need to submit a bug report so that it gets fixed.

4.5.1 Bug in the R-Core supported packages, their documentations, and/ or in the R language

1. Packages that are supported by the R-Core are labelled with `Maintainer: R Core Team <R-core@r-project.org>`. One simple way to get the information from R is by running the `maintainer("package_name")` command.
2. The bug report for R-Core supported packages, their documentations, and/ or a bug report for the R language itself can be submitted either to R-devel, see posting guide, or to Bugzilla. In the future, we hope to have an option to report an issue to the GitHub Mirror of the R svn server.
3. In order to report bugs, as well as fixes, using Bugzilla, please ensure that you have a Bugzilla account
4. Please ensure whether the bug is already fixed (in the upcoming changes in R) or reported (search for it from those already reported on Bugzilla, either on search existing bug reports, using the advanced search option here, or show open bugs new-to-old).

4.5.2 Bug in the non R-Core supported packages and/or their documentations

For packages that are not maintained by the R-Core, the bug reports can be submitted at, perhaps, an issues tracker url on GitHub/GitLab/R-Forge. To find if such an issues tracker is available, you can look at the package `DESCRIPTION` file first (e.g. using `packageDescription("package_name")`) to check if a url is provided in the `BugReports` field. If that is not available, then the package maintainer can be contacted (using `maintainer("package_name")`). In R running the function `bug.report(package = "package_name")` shall direct you to either the GitHub issue tracker of the package, or

to the bug tracking web page, or towards composing an e-mail to the package maintainer. This function `bug.report` is disabled in RStudio, by default. However, if you use `utils::bug.report(package = "package_name")` then it works on RStudio as well. Please ensure that the bug is not already reported or fixed before reporting it in any of the ways suggested above.

4.5.3 Bug in a documentation website managed by the R Core Team

The R Core Team maintain a selection of websites that each have their own ways for reporting issues outside of Bugzilla. Have a read through the posting guide for R project mailing lists and then follow the instructions below to give feedback to the maintainers.

- R FAQ contains answers to some of the most frequently asked questions about R. Feedback on the R FAQ should be sent to the R-devel mailing list via R-devel@R-project.org.
- R for Windows FAQ contains answers specific to the Windows port of R. Feedback on the Windows FAQ should be sent to the R-SIG-windows mailing list (the Special Interest Group for Windows), via R-windows@R-project.org.
- R for macOS FAQ contains a user guide to the R.APP version of R and information on using R on macOS, supplementing the main R manuals. Feedback on the macOS FAQ should be sent to the R-SIG-Mac mailing list (the Special Interest Group for Mac ports of R), via r-sig-mac@r-project.org.
- Feedback on the R Project website should be sent to webmaster@r-project.org.
- Feedback on the website for The Comprehensive R Archive Network (CRAN) should be sent to cran-sysadmin@r-project.org.

4.6 Good practices in reporting bugs / Expectations of a good bug report

If you follow the practices given below, you will come up with a good bug report which might make it easier for the maintainer(s) to fix the bug.

1. Include a minimal reproducible example of the bug in your report. The maintainer should be able to quickly reproduce the bug using the minimal example that you provide. Here is a community wiki post on how to make a minimal reproducible example.
2. Mention the software architecture on which the bug occurred.
3. Use inbuilt data sets as far as possible.

In addition to the above, here are the bug writing guidelines on Bugzilla. The bug reporting documentation in R also discusses practices to write a good bug report.

Once you have successfully reported a bug, you will likely receive an update each time an action is taken on the bug. On Bugzilla, the report may be given one of the following status: New, Assigned, Confirmed, Reopened, Unconfirmed.

4.7 Disagreement with a resolution on the bug tracker

As humans, there might be differences of opinions from time to time. What needs to be considered here is to be respectful of the fact that care, thought, and volunteer time has gone into the resolution of the issue or bug.

If you take some time, then on reflection, the resolution steps may seem more reasonable than you initially thought. If you still feel that the resolution is incorrect, then raise a thoughtful question to the person who resolved it. If the issue was not carefully thought about in the first place then it is less likely to win any conversion of thought.

As a reminder, issues closed by a core developer on Bugzilla have already been carefully considered. Please do not reopen a closed issue. Although one can comment on a closed issue, if necessary. Every comment on an issue generates an email to every R-core member (unless they have the notifications disabled). So it would be best to be considerate while commenting on issues, especially in case of closed issues or when you are commenting in pure agreement without adding anything beyond that to a discussion (the +1 type posts which are perfectly acceptable in other contexts).

4.8 Examples of Bug reports submitted on Bugzilla or R-devel mailing list

If you like to see how bugs are reported on Bugzilla, here are some examples:

1. Bug 17767 - Warning about incomplete argument within nlme: A bug report with a reproducible example, a patch, and a review.
2. Possible bug when finding shared libraries during staged installation: A bug report submitted by Kara Woo which was promptly fixed via the R-devel mailing list. (More information about the R-devel mailing list can be found here).
3. Bug: time complexity of substring is quadratic as string size and number of substrings increases: A substring reported by Toby Dylan Hocking and fixed by Tomas Kalibera, Feb 2019 via the R-devel mailing list.
4. patch for gregexpr(perl=TRUE): A gregexpr bug report and patch submitted by Toby Dylan Hocking and merged by Tomas Kalibera, Feb 2019 via the R-devel mailing list.

4.9 See also

1. Reporting a bug
2. R FAQ on bugs
3. Bugzilla guidelines of reporting a bug

5 Submitting Feature Requests

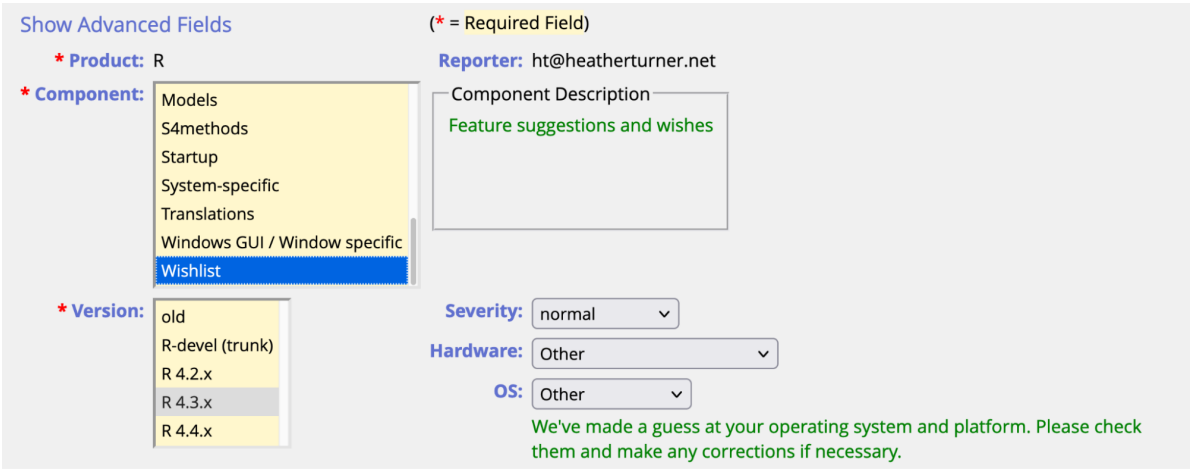
5.1 What is a feature request?

A feature request is a suggestion or proposal to add new functionality or improve an existing feature.

We recommend sharing your feature request idea on the R-devel mailing list and gaining support from R-Core members and other users to increase its likelihood of acceptance.

5.2 How to submit a feature request?

You can submit a feature request by filing a bug on Bugzilla. Under Component, select "Wishlist", the designated label for feature requests.



Similar to bugs, you should ensure that the feature request is not already reported and follow best practices whenever possible.

5.3 Examples of feature requests submitted on Bugzilla

If you like to see how feature requests are reported on Bugzilla, here are some examples:

1. Bug 18690 - Increase CONSOLE_BUFFER_SIZE and / or warn if input is truncated: A feature request to ease the readability of long lines.
2. Bug 17912 - ordering elements in sessionInfo() by alphabetical order: A feature request that was not accepted due to the prioritization of load time in session information.

6 Reviewing Bugs

6.1 How you can help to review bug reports?

After understanding where bugs are reported in R (Bugzilla) or in other projects (GitHub/GitLab/R-Forge), a great way to contribute is reviewing bug reports.

Around the clock, new bug reports are being submitted on Bugzilla or the bug trackers (for instance, GitHub issues) of R packages and existing bug reports are being updated. Every bug report needs to be reviewed to make sure various things are in proper order. You can help with this process of reviewing bugs.

6.1.1 Preparing to review bug reports

If you want to review bug reports on Bugzilla, you are required to have a Bugzilla account. More details on how you can review a bug report are available on this post on the R Blog: R Can Use Your Help: Reviewing Bug Reports

6.2 Classifying bug reports

A good bug report is the one which:

1. Explains clearly how to reproduce the bug.
2. Includes the version of R, the machine architecture, and the operating system platform on which the bug occurred.

Relevant details should be a part of a good bug report. You can help with the following tasks once you have some R programming experience:

1. Reproducing the bug: If you see a bug report which does not clearly explain how to reproduce it, you can try reproducing the bug and eventually make things easier for the core developer(s) and/or package maintainer(s).
2. Checking different binary builds: Check whether the bug occurs on a different binary build of R. It is helpful to know whether the bug is affecting: `r-patched`, `r-devel`, or `r-release` binary builds of R.

3. Writing a unit test: If the bug report lacks a unit test that should be a part of R's test suite, then you can help with providing it.

These helpful tasks allow the Core developers and/ or maintainers to classify a bug report properly, so that the bug can be handled in a timely fashion.

6.3 How to find a bug report or an issue to review?

1. You may search old bug reports or issues that could be closed. Old bug reports may no longer be valid or may include a patch that is ready to be committed, but no one has had the time to review and commit.
2. You might also want to search for issues in topics in which you have a working knowledge. When on Bugzilla you can use the advanced search to find specific topics. Bug reports are by default public on Bugzilla (unless the defaults are changed to avoid security vulnerability).

6.4 Example of a bug review submitted on Bugzilla

If you would like to see how bugs are reviewed on Bugzilla, the Bug 16542 - `nlme:::summary.lmList` with unequal outputs per group is an example where an old bug report is being reviewed. It is tested to see if it was still an issue and a few ways are proposed to resolve the issue.

> **Note**
>
> There is a `#bug-reporting` channel on the R Contributors slack where you can share your bug report(s) for review/feedback before submitting to Bugzilla. This can help with checking that it really is a bug, that you have included the important information and excluded redundant information.

6.5 See also

1. Reviewing bug reports: Blog

7 Finding the Source

This chapter discusses how you can have an overview of the R codebase. For instance, where to find the implementation of a base function written in R and where to find a primitive implementation written in C. You may want to find the source code of a function just out of curiosity or maybe to gain more insight into what a particular function is actually doing. Whatever be the case, reading the source code will help you to learn a lot about any function.

7.1 Finding R source code

1. Find the R function with the code of interest. You will always be able to print the top-level function (or use `View(function_name)` in RStudio). Looking at the code for the body of this function will reveal what you need to do next:

- Can already see code of interest: stop here or skip to step 3 to find the corresponding file in the R sources.
- Code of interest is in nested R function: go to step 2.
- Top-level function is an S3 generic, identified by a call to `UseMethod()`. Use `methods(function_name)` to see available methods, then go to step 2.
- Code of interest is in compiled code, identified by a call to `.C()`, `.Call()`, `.Fortran()`, `.External()`, and `.External.graphics()`, or `.Internal()` and `.Primitive()`: go to section on compiled code.

2. Nested functions or S3 methods may not be exported by the package they are in. If this is the case, the simplest way to view the code is to use `getAnywhere()` or `getS3method()`. Now you can keep looking at nested R functions till you find the code of interest or hit a call to compiled code.

3. Find an R function in the R sources. Two options here:

- Search on the internet: For R Core packages, search on the GitHub mirror (https://github.com/r-devel/r-svn); for recommended packages, use the CRAN mirror (https://github.com/cran) - this will link to the source on GitHub if available, e.g. https://github.com/cran/survival. Note that GitHub search ignores wildcard characters

```
. , : ; / \ ` ' " = * ! ? # $ & + ^ | ~ < > ( ) { } [ ]
```

but this does not include - so you can search for a function or S3 method as follows:

```
"body <- function" extension:R
"quantile.ecdf <- function" extension:R
```

- Search in the R sources using grep: The R Patched and Development Versions chapter discusses how to download the R sources directly or from the svn repository. Now if the sources are in `~/R-devel`, you can search as follows:

```
grep -R "body <- function" ~/R-devel/src
grep -R "quantile <- function" ~/R-devel/src/library
```

Note: The above procedure does not cover S4, R6 or RC generics or methods. Refer accessing R source for further details.

7.2 Finding C source code

1. If `.Internal()` or `.Primitive()`, find entry point in `names.c` as described in the Jenny Bryan's post of accessing R source. For all other calls to compiled code, you can find the entry point from within R. For instance, the body of complete.cases() is

```
.External(C_compcases, ...)
```

`C_compcases` inherits from class "NativeSymbolInfo" and we can extract the name of the entry point via

```
stats:::C_compcases$name
```

We know that it is in the stats package as we see that when we print complete.cases or look at the help file. This shows us that the entry point is simply "compcases" and in fact that is the general convention in R code, that you simply remove the `C_` prefix (sometimes `.F_` for Fortran code[1]) in the name of the object passed to the first argument of the call.

2. Once you have the entry point, search as for R code. In the case of searching on GitHub, restrict the search to files with the relevant extension

[1]Other variations are also possible. For example, `.External.graphics()` calls to C functions in the grid package have a `C_` prefix in the R code, but an `L_` prefix in the C code.

```
compcases path:*.c
lowesw path:*.f
```

similarly for grep

```
grep -R --include=*.c "compcases" ~/R-devel/src/library/
```

Note:

1. Many editors (like RStudio, ESS) support ctags for code browsing, making it easy to jump to definitions of functions. `R CMD rtags` can generate ctags for any R code (Credit: Deepayan Sarkar).
2. A more sophisticated system is called GNU GLOBAL, which also supports finding all references (calls) to a function.
3. GitHub has a code navigation feature via the library tree-sitter. Unfortunately, it does not have R support yet. An R driver for tree-sitter made by Jim Hester is available.

7.3 See also

Read the R source blogpost.

8 Lifecycle of a Patch

8.1 Introduction

R uses a workflow based on patches. A patch is the set of differences (additions and deletions) between two versions of code. So you can create a patch defining a bug fix or a proposed update to the R codebase and submit it through your official Bugzilla account to the core developer(s). Be clear in your communication as it is the key to contributing to any project, especially an open source project like R.

8.2 When do you submit a patch?

There might be a situation where you come across a bug in R, which you may have an idea of how to fix. This can turn out to be an opportunity for you to submit a patch. By submitting a patch or a bug fix, you are helping to reduce the workload on the R developers in addition to yourself being a contributor to R!

When you submit a patch, you are helping the developer(s) and maintainer(s) so that they do not have to write the entire code from scratch. Instead, they can test and tweak your patch, if necessary.

8.3 What tools are required to submit a patch?

To submit a patch, you need:

1. SVN installed on your machine.
2. The latest developer version of R.

You can retrieve the latest source code of R via:

```
export TOP_SRCDIR="$HOME/Downloads/R"
svn checkout https://svn.r-project.org/R/trunk/ "$TOP_SRCDIR"
```

Depending on the operative system you might need to do some steps before that. The different steps required can be found in previous chapters of the book, for Windows, macOS and Linux.

8.4 How to prepare a patch?

If you have the source code in `$TOP_SRCDIR` you can edit the files, for example a documentation file such as `"$TOP_SRCDIR"/src/library/stats/man/Multinom.Rd`, to make your desired changes to that or more files.

Then you should check that R still works as expected via:

```
cd "$TOP_SRCDIR"
make check-devel
```

If there is no test for your proposed change you can add a new regression test, following the guidelines.

Then you should bring changes from the repository into the working copy, in case any other change has been introduced, and create a patch.diff file with just the changes you want to propose to the R core:

```
svn update
svn diff > patch.diff
```

Most often, changes are made to existing files, but if you happen to be adding a new file in your change, you'll need to run `svn add path/to/file1 ...` before running `svn diff` to mark those files for inclusion.

This `patch.diff` file is the one that can be proposed to the R core via Bugzilla. You can also ask for reviews to the patch before proposing it to the R core via the r-devel mailing list or the slack channel of the R-contributors space.

8.4.1 Using a git mirror

Besides checking in your computer, you can use the Github mirror r-devel/r-svn of the source code to check this patch with different configurations and OS.

You should first find the file to edit, via the github interface for example:

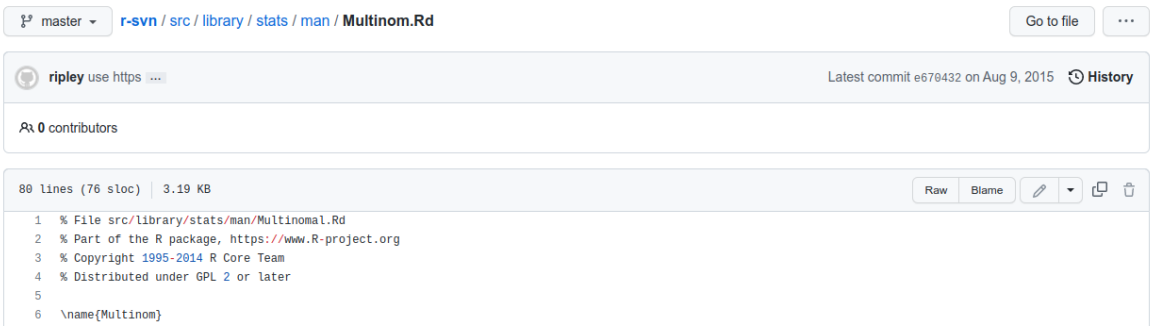


Figure 8.1: Screenshot of the heading of the src/library/stats/man/Mulinom.Rd

Then you can edit it, directly in the interface or using the github interface:

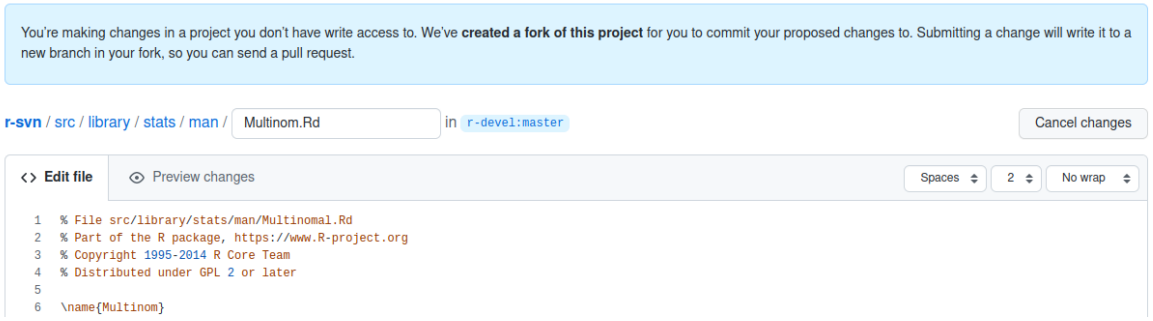


Figure 8.2: Screenshot of the file src/library/stats/man/Mulinom.Rd being edited via the Github interface

Create a commit with a message describing the changes

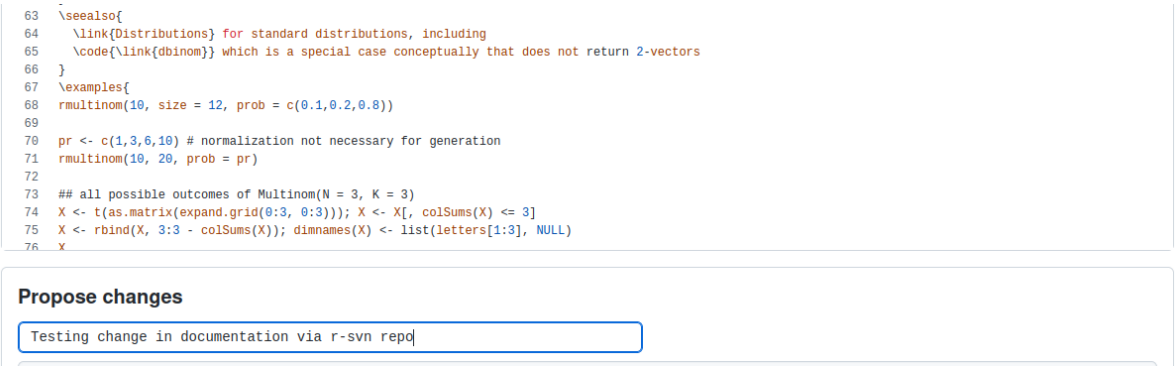


Figure 8.3: Screenshot of the commit message

And create a pull request from the branch created to check the changes.

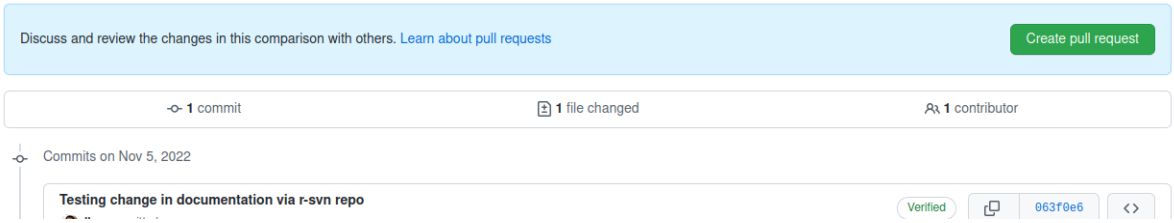


Figure 8.4: Screenshot of the message when opening a pull requests from the branch

Add a message and description of the svn for other users and the R core to know what is the purpose of this modification:

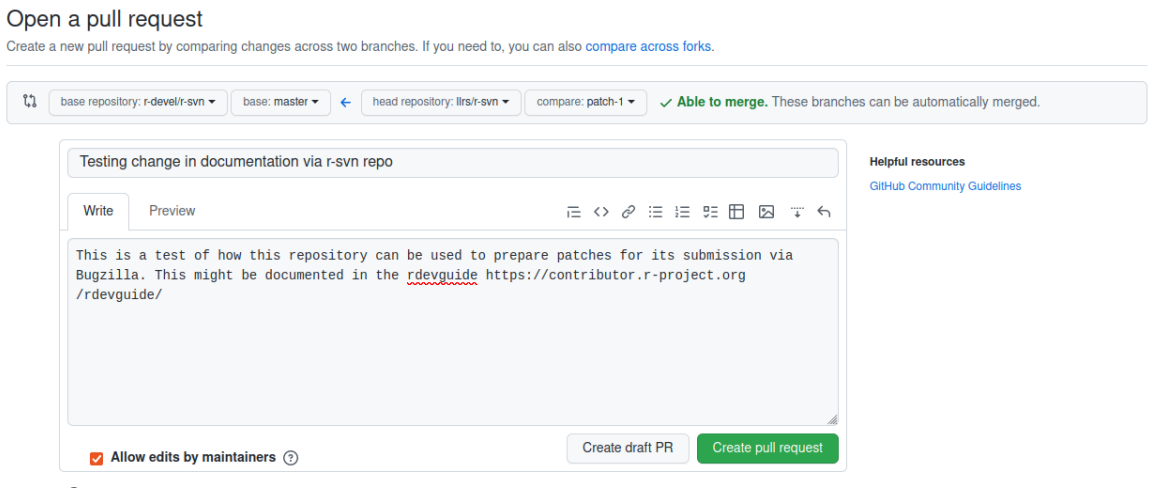


Figure 8.5: Screenshot of the message and content while opening a pull requests for the r-svn repository

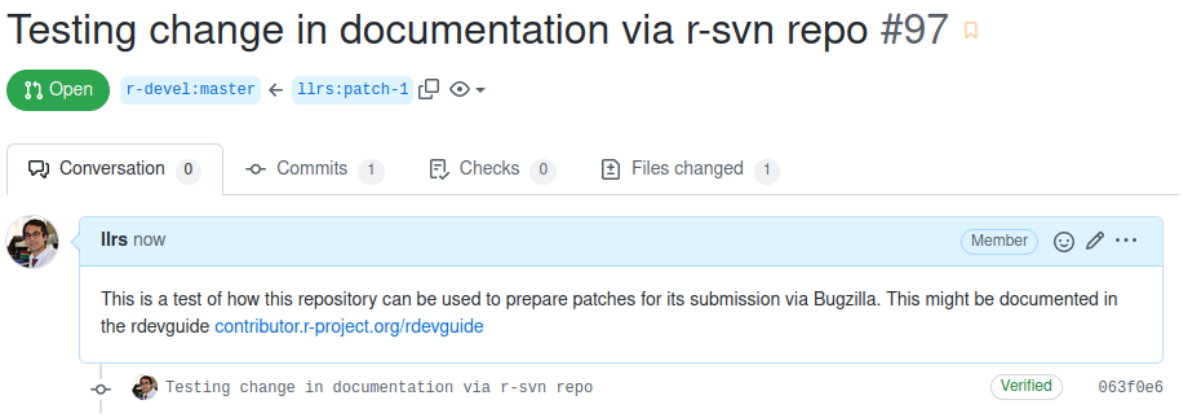
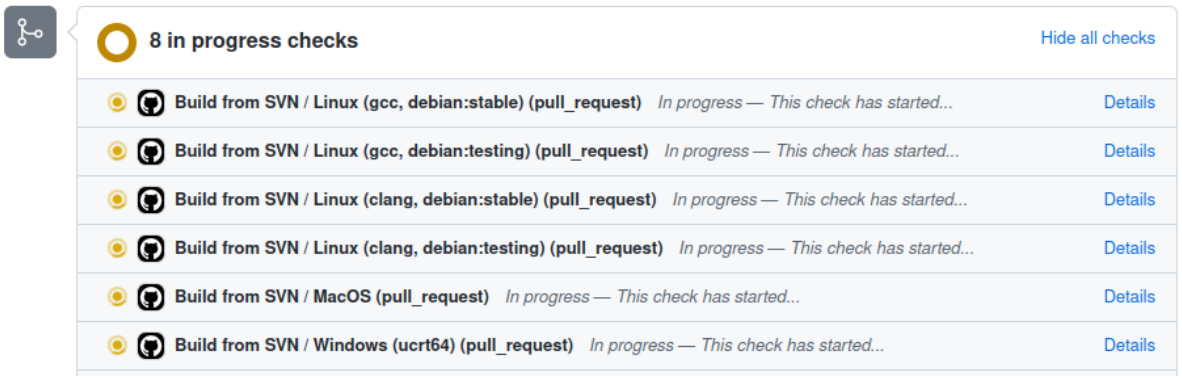


Figure 8.6: Screenshot of the pull requests opened

Once the PR is submitted, some automatic checks will be triggered (they might need to be approved by some other users as per Github rules):



When the checks end you will need to explore the results and asses if the results indicate a problem or not.

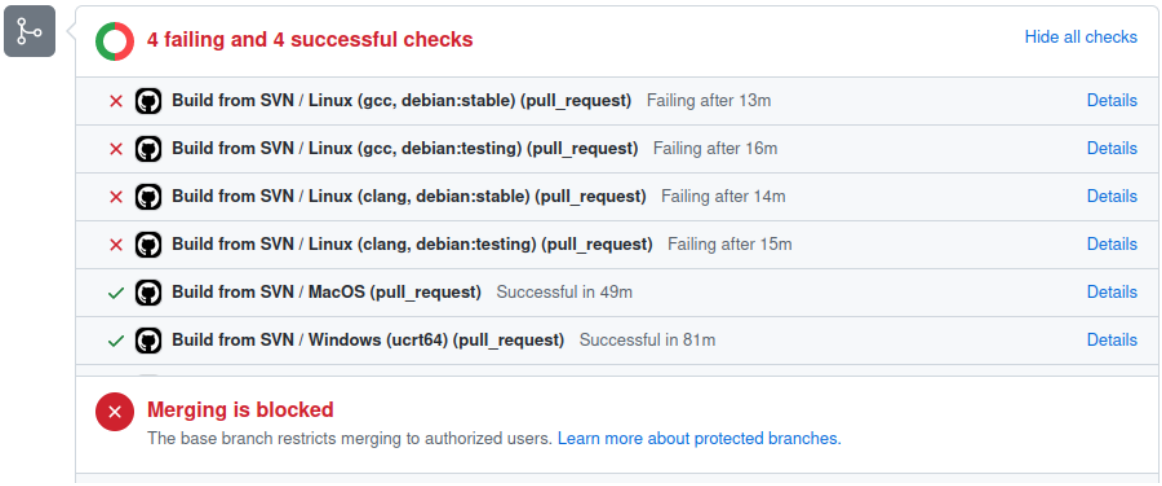


Figure 8.7: Screenshot of the results of the Github checks in the r-svn mirror

Once you are happy with the changes and the checks report that everything is okay you can retrieve the patch via:

`https://patch-diff.githubusercontent.com/raw/r-devel/r-svn/pull/<pull_request_number>.diff`

Save `<pull_request_number>.diff` as a plain text file to submit your patch, remember to check if it meets the recommendations for good patches.

If you want to use `git` from the terminal to create the pull request (PR) to test the changes, you can use this summary of the available git commands.

8.5 Making good patches

When creating a patch for submission, there are several things that you can do to help ensure that your patch is accepted:

1. Make sure to follow R's coding standards (R is a GNU project and there are GNU coding standards). The coding style of the patch you submit should largely match with the codebase it is being applied to. If your patch has one or two minor discrepancies, then those may be fixed by the core developer who will eventually test your patch. However, if there are systematic deviations from the style guides your patch will be put on hold until you fix the formatting issues. There is no comprehensive official R style manual, however some nearly universal standards are summarised in this article.

2. Be aware of backwards-compatibility considerations. While the core developer who eventually handles your patch will make the final call on whether something is acceptable, thinking about backwards-compatibility early will help prevent having your patch rejected on these grounds. Put yourself in the shoes of someone whose code will be broken by the change(s) introduced by the patch. It is quite likely that any change made will break someone's code, so you need to have a good reason to make a change as you will be forcing someone to update their code. This obviously does not apply to new functions or new arguments. New arguments should be optional and have default values which maintain the existing behaviour. If in doubt, discuss the issue with experienced developers.
3. Make sure you have proper tests to verify that your patch works as expected. Patches may not be accepted without the proper tests.
4. Make sure the entire test suite runs without failure because of your changes. It is not sufficient to only run whichever test seems impacted by your changes, because there might be interactions unknown to you between your changes and some other part of the interpreter.
5. Proper documentation additions/changes should be included.
6. Each bugfix should ideally be addressed by a single patch. In particular, do not fix more than one issue in the same patch (except, if one code change fixes all of them) and do not do cosmetic changes to unrelated code in the same patch as some bugfix.

8.6 Submitting your patch for review

8.6.1 Patch in response to a pre-existing issue or bug report

In this case, you should attach the patch to the existing issue or bug report on Bugzilla with a brief comment.

1. Use the Attachments option to add the `*.diff` file as attachment.
2. On the new Create New Attachment web page, add a Brief Description.
3. Select Content Type as `patch`.
4. Add comments (often prose text) in this page rather than in the original bugzilla page for the PR.
5. Press Submit. This will give you a bugzilla submission that sends one e-mail to all of R-core plus the PR author.

8.6.2 Patch in response to an unreported issue or bug report

Assuming you already performed a search on Bugzilla for a pre-existing issue or bug and did not find the issue or bug reported, you need to create a new bug report and include your patch with it. Please fill in as much relevant detail as possible to prevent reviewers from having to delay reviewing your patch because of lack of information. Include (mostly as the first sentence), a to-the-point explanation of what the purpose of the patch is. This sentence should not be in the descriptive form, rather an imperative form will be more suitable here. If this is not enough detail for a patch, a new paragraphs(s) can be added to explain in proper depth what has happened. The details should be good enough that a core developer reading it understands the justification for the change.

8.7 Getting your patch reviewed

To begin with, please be patient. There are many more people submitting patches than there are people capable of reviewing your patches. Getting your patch reviewed requires a reviewer to have the spare time and motivation to look at your patch. We cannot force anyone to review patches and no one is employed to look at patches.

There is a `#patches-for-review` channel on the R Contributors slack where you can share your patch(es) for review/feedback before submitting to R-Core/Bugzilla. This can help with checking that you have included the important information and excluded redundant information.

If your patch has not received any notice from reviewers (i.e., no comment made) after one month, comment/message on the `#patches-for-review` channel to remind the members that the patch needs a review.

When someone does manage to find the time to look at your patch they will most likely make comments about how it can be improved. It is then expected that you update your patch to address these comments, and the review process will thus iterate until a satisfactory solution has emerged.

8.7.1 How to review a patch?

One of the bottlenecks in the R development process is the lack of code reviews. If you browse Bugzilla, you will see that numerous issues have a fix, but cannot be merged into the main source code repository, because no one has reviewed the proposed solution. Reviewing a patch can be just as informative as providing a patch and it will allow you to give constructive comments on another developer's work. This guide provides a checklist for submitting a patch review. It is a common misconception that in order to be useful, a patch review has to be

perfect. This is not the case at all. It is helpful to just test the patch and/or play around with the code and leave comments in the patch or on Bugzilla.

If a bug report or an issue has a patch attached that has not been reviewed, you can help by making sure that the patch:

- follows the style guides;
- is a good solution to the problem it is trying to solve;
- includes proper tests; and
- includes proper documentation changes.

Also refer to Making good patches for more ideas. Doing all of this allows the core developer(s) and/ or maintainer(s) to more quickly look for subtle issues that only people with extensive experience working on R's codebase will notice.

8.8 Leaving a patch review on Bugzilla

When you review a patch, you should provide additional details and context of your review process and leave comments. For example:

1. If you tested the patch, report the result and the system and version tested on, such as 'Windows 10', 'Ubuntu 16.4', or 'Mac High Sierra'.
2. If you request changes, try to suggest how or attach an updated patch.
3. Comment on what is 'good' about the patch, not just the 'bad'. Doing so will make it easier for the patch author to find the good in your comments.

8.9 Dismissing review from another core developer

A core developer can dismiss another core developer's review if they confirmed that the requested changes have been made. When a core developer has assigned the patch to themselves, then it is a sign that they are actively looking after the patch, and their review should not be dismissed.

8.10 Acceptance or rejection of your patch

Once your patch has reached an acceptable state, it will either be applied or rejected. If it is rejected, please do not take it personally. Your work is still appreciated regardless of whether your patch is applied. Balancing what does and does not go into R is tricky and everyone's contributions cannot always be accepted.

But if your patch is accepted and applied it will then go on to be released with the next patched release and eventually the next major release of R. It may also be backported to older versions of R as a bugfix if the core developer doing the patch acceptance believes it is warranted.

It may take longer before your patch is accepted and applied or rejected, sometimes even months or years. Nonetheless, it is appreciated that you submitted a patch.

8.11 Examples of patch reports on Bugzilla

8.12 Examples of reviewing a patch

8.13 See also

1. Submitting patches

9 Documenting

R has a substantial body of documentation, comprising help files for the core packages and a set of manuals aimed at users (An Introduction to R, R Data Import/Export), system administrators (R Installation and Administration) and developers (Writing R Extensions, The R language definition, and R Internals). This documentation was developed by the R Core Team with input from external contributors. The continuing involvement of the user community is important in maintaining this valuable documentation.

The involvement of the community takes many forms, from contributing content, to making bug reports or raising an issue when the documentation could be more complete or made easier to follow.

This chapter is about the ways people can contribute to R's documentation, with guidance on how to do that. Any time you feel that you can clarify or fill gaps in existing documentation, your contribution is welcome and appreciated. If you find it difficult to deal with the markup formatting language used in the source files, you can ask for help with that part too. Please do not let the material in this chapter stand between the documentation and your desire to help out. However, not every good faith effort to change or extend the documentation will be accepted - sometimes the suggested changes may be incorrect; other times, while a change in wording may make some things clearer and easier to understand, the finer details of some corner case may become less clear, leading to the suggested changes being declined or modified by a member of R Core before applying them (if they agree the issue is important enough to fix).

9.1 Helping with documentation

Maintaining the accuracy of R's documentation and keeping a high level of quality takes a lot of effort. Community members, like you, help with writing, editing, and updating content, and these contributions are appreciated and welcomed.

Looking at pre-existing documentation source files can be very helpful when getting started.

You can directly search for documentation issues/bugs on Bugzilla. Issues vary from typos to unclear documentation and items lacking documentation.

If you see a documentation issue that you would like to tackle, you can leave a comment on the issue saying you are going to try to solve the issue and mention roughly how long you think

you will take to do so (this allows others to take on the issue if you happen to forget or lose interest).

9.2 Reporting documentation bugs

To suggest improvements to the R manuals, report typos or bugs in the R manuals, or to raise issues related to documentation in packages maintained by the R Core Team, follow the usual guidance for reporting bugs in R.

To report bugs or typos in the documentation of a package that is not maintained by the R Core Team, follow the guidance for reporting bugs in contributed packages.

Note:

There is a `#core-documentation` channel on the R Contributors slack where you can get feedback before reporting an issue or get feedback on proposed improvements.

9.2.1 Example documentation bugs

Here are a couple of successfully resolved bugs as examples.

1. Bug 16003 - heatmap scale argument: description not complete

There was a question about the description in the documentation, where it wasn't technically incorrect, but was slightly unclear. This led to a discussion around the updates to the text and a couple of extra examples were added in for clarity too.

2. Bug 18781 - typo in texinfo

This is an example of a typo being spotted, raised and fixed within the same day.

9.3 Guidelines for writing R help files

This section is based on the guidelines used by R Core developers for writing R help files. Extensive details of writing R documentation files can be found in the Writing R Extensions manual.

The language used in the documentations should follow these basic rules:

1. Affirmative tone should be used to describe what the function does and how to use it effectively. Rather than creating worry in the mind of a reader, it should establish confident knowledge about the effective use of the particular function/feature.

2. More documentation is not necessarily better documentation. Long descriptions full of corner cases and caveats can create the impression that a function is more complex or harder to use than it actually is. Be succinct but exhaustive.

3. Short code examples can help in understanding better. Readers can often grasp a simple example more quickly than they can digest a formal description. Usually people learn faster with concrete, motivating examples that match the context of a typical use case.

4. Giving a code equivalent (or approximate equivalent) can be a useful addition to the description provided. You should carefully weigh whether the code equivalent adds value to the document.

5. The tone of the documentation needs to be respectful of the reader's background. Lay out the relevant information, show motivating use cases, provide glossary links, and do your best to connect-the-dots. The documentation is meant for newcomers, many of whom will be using it to evaluate the R language as a whole. The experience needs to be positive and not leave the reader with worries that something bad will happen if they make a mistake.

9.4 Introduction to `.Rd` files

The R help files are written in "R documentation" (Rd) format, a markup language which resembles LaTeX. The `.Rd` file format can be further processed into a variety of formats, including LaTeX, HTML, and plain text. The `.Rd` files can be found in the `man` directory of the source code for the corresponding package.

There are three main parts of an `.Rd` file:

1. **Header**: This part is for the basic information of the document/file. For instance, the name of the file, the topics documented, a title, a short textual description, and R usage information for the objects documented.

2. **Body**: This part includes further information on the function's arguments and return value.

3. **Footer**: This part is optional. Usually the keyword information is included here.

All the above information is included in a `.Rd` file within a series of sections with standard names (user-defined sections are also allowed). These sections are discussed below:

1. `\title` section:

    - Capitalize each word.
    - Do not end in a period.
    - Avoid use of markup language (because markup language need not be suitable for various hypertext search systems).

2. `\usage` and `\examples` sections:

    - Line length of 65 characters is advised.
    - Use `TRUE` instead of `T` and `FALSE` instead of `F`.
    - Add spaces around binary operators.
    - Add spaces after commas in the argument lists.
    - Use `<-` rather than `=` for assignments.
    - Add spaces around the `<-` operator.
    - Do not use tabs to indent (as these do not render correctly on all possible pagers).
    - Use 4 spaces to indent the (example) code.
    - Make sure the examples are directly executable.
    - The examples should be system-independent.
    - The examples should not require special facilities (for instance, Internet access or write permission to specific directories).
    - Examples should also not take longer than necessary to run, as they are run when checking a build of R.

3. `\source` and `\references` sections:

    - Author(s) names should be written in the form `Author, A. B.`.
    - Author(s) names should be separated by a comma or `and` (but not both).
    - Separate paragraphs (separated by a blank line) should be used for each reference.
    - Give a date immediately after the author(s) names.
    - Do not put a period after the date.
    - Titles of books and journals (not articles) should be enclosed in `\emph{...}`.
    - Volume numbers for journals are to be enclosed in `\bold{...}` and followed by a comma.
    - Use `--` for page ranges.
    - For giving an address for a publisher use the format `New York: Springer-Verlag`.

For example, the help file for `base::mean()` is found at https://svn.r-project.org/R/trunk/src/library/base/man/mean.Rd. The file`mean.Rd` has the content shown below:

```
% File src/library/base/man/mean.Rd
% Part of the R package, https://www.R-project.org
% Copyright 1995-2022 R Core Team
% Distributed under GPL 2 or later

\name{mean}
\title{Arithmetic Mean}
\usage{
mean(x, \dots)

\method{mean}{default}(x, trim = 0, na.rm = FALSE, \dots)
```

```
}
\alias{mean}
\alias{mean.default}
\arguments{
  \item{x}{An \R object.  Currently there are methods for
    numeric/logical vectors and \link[=Dates]{date},
    \link{date-time} and \link{time interval} objects.  Complex vectors
    are allowed for \code{trim = 0}, only.}
  \item{trim}{the fraction (0 to 0.5) of observations to be
    trimmed from each end of \code{x} before the mean is computed.
    Values of trim outside that range are taken as the nearest endpoint.
  }
  \item{na.rm}{a logical evaluating to \code{TRUE} or \code{FALSE}
    indicating whether \code{NA} values should be stripped before the
    computation proceeds.}
  \item{\dots}{further arguments passed to or from other methods.}
}
\description{
  Generic function for the (trimmed) arithmetic mean.
}
\value{
  If \code{trim} is zero (the default), the arithmetic mean of the
  values in \code{x} is computed, as a numeric or complex vector of
  length one.  If \code{x} is not logical (coerced to numeric), numeric
  (including integer) or complex, \code{NA_real_} is returned, with a warning.

  If \code{trim} is non-zero, a symmetrically trimmed mean is computed
  with a fraction of \code{trim} observations deleted from each end
  before the mean is computed.
}
\references{
  Becker, R. A., Chambers, J. M. and Wilks, A. R. (1988)
  \emph{The New S Language}.
  Wadsworth & Brooks/Cole.
}
\seealso{
  \code{\link{weighted.mean}}, \code{\link{mean.POSIXct}},
  \code{\link{colMeans}} for row and column means.
}
\examples{
x <- c(0:10, 50)
xm <- mean(x)
c(xm, mean(x, trim = 0.10))
```

```
}
\keyword{univar}
```

Many R package developers write help files using the R package roxygen2, which generates `.Rd` files from comments in the corresponding `.R` files. However, in this guide we only consider `.Rd` files, because the help files for the base distribution are written and edited directly in `.Rd` format.

9.5 R manuals

While R help files use the `.Rd` format, the official R manuals are written in GNU Texinfo (`.texi` files).

- **Editing:** Since `.texi` files are plain text, you can edit them with any text editor.
- **Compiling:** To see how your changes look, you can "compile" them into PDF or HTML using tools like `makeinfo` or `texi2any` (both part of the GNU Texinfo distribution).
- **Source:** You can find the source files for these manuals in the `doc/manual` directory of the R source code.

Improvements can be suggested via the R-devel mailing list or the bug tracker even without a formal patch.

The R manuals are a part of the R sources. Hence, bug reports/patches can also be submitted via Bugzilla, e.g. Bug 15221 - R-admin/'Installing R under Windows': Missing argument name. Note that they are typically referred to by their file names as listed below:

Manual	Nickname
An Introduction to R	"R-intro"
R Data Import/Export	"R-data"
R Installation and Administration	"R-admin"
Writing R Extensions	"R-exts"
The R language definition	"R-lang"
R Internals	"R-ints"

> **Note**
>
> - Every manual is associated with a particular version of R, so you should check the version before reporting a bug.
> - The R manuals page has links for the three types of release: `r-release`, `r-patched` and `r-devel`. These nicknames appear in the URLs, e.g. https://cran.r-project.org/doc/manuals/r-release/R-intro.html.

- The Texinfo manual should be referred to for how to mark up text.

9.6 Proofreading

While an issue filed on Bugzilla means there is a known issue somewhere, that does not mean there are not other issues lurking about in the documentation. Proofreading a part of the documentation can often uncover problems.

If you decide to proofread, read a section of the documentation from start to finish, filing issues in Bugzilla for each major type of problem you find. It is best to avoid filing a single issue for an entire section containing multiple problems; instead, file several issues so that it is easier to break the work up for multiple people and more efficient review.

9.7 Helping with the R Development Guide

The R Development Guide (what you are reading now) uses the same process as the main R documentation, except for some small differences. The source lives in a GitHub repository and bug reports should be submitted to the devguide GitHub tracker.

There's a section on the introduction page detailing how to contribute to this guide. Once you have made suggested changes you can raise them for someone else to review through a pull request on GitHub. Examples of past pull requests can be found on the pull request tab in this guide's GitHub repository. Our workflow for the R Development Guide uses continuous integration and deployment so changes to the live R Development Guide site are normally published once a pull request is merged.

9.8 See also

1. Writing R documentation files

10 Message Translations

This chapter covers internationalization in R, i.e., the display of messages in languages other than English. All output in R (such as messages emitted by `stop()`, `warning()`, or `message()`) is eligible for translation, as are menu labels in the GUI. Depending on the version of R that you are using, some of the languages might already be available while others may need work. R leverages the `gettext` program to handle the conversion from English to arbitrary target languages.

Having messages available in other languages can be an important bridge for R learners not confident in English – rather than learning two things at once (coding in R and processing diagnostic information in English), they can focus on coding while getting more natural errors/warnings in their native tongue.

The `gettext` manual is a more canonical reference for a deep understanding of how `gettext` works. This chapter will just give a broad overview, with particular focus on how things work for R, with the goal of making it as low-friction as possible for developers and users to contribute new/updated translations.

10.1 How translations work

Each of the default packages distributed with R (i.e., those found in `./src/library` such as `base`, `utils`, and `stats` and which have priority base) contains a `po` directory. A `po` directory is the central location for cataloguing/translating each package's messages. It contains a template message file (`.pot`) for the corresponding package along with translated `.po` files (that are created using the template `.pot` file).

10.1.1 `.pot` files

A `.pot` file is a template file found inside the `po` directory of an R package. This template file is a snapshot of the messages available in a given **domain**. A domain in R typically identifies a source package and a source language (either R or C/C++). For example, the file `R-stats.pot` (found in the R sources in `./src/library/stats/po`) is a catalogue of all messages produced by R code in the `stats` package, while `stats.pot` is a catalogue of all messages produced by C code in the `stats` package.

The 'base' package has two exceptions to the basic pattern described above. The first is the domain for messages produced by the C code which is the fundamental backing of R itself (especially, but not exclusively, the C code under ./src/main[^The file `./po/POTFILES` is the canonical source of files searched. Note that while, technically, it is possible to support translations in Fortran code, R does not currently do so. Only a handful of messages are produced by Fortran routines in the R sources.]). The associated `.pot` file is `R.pot` and is found in `./src/library/base/po`. R-base.pot`is a normal`.pot`file because base has a normal`R' directory.

The second is the domain for the Windows R GUI, i.e., the text in the menus and elsewhere in the R GUI program available for running R on Windows. These messages are stored in the `RGui.pot` domain, also in the `po` directory for `base`, and are most commonly derived from C code found in `./src/gnuwin32`. One reason to keep this domain separate is that it is only relevant to one platform (Windows). In particular, Windows has historically different character encodings, so that it made more sense for Windows developers to produce translations specifically or Windows, since it is non-trivial for non-Windows users to test their translations for the Windows GUI.

10.1.1.1 Generating `.pot` files

For outside contributors, there's no need to update .pot files – translators will typically take the R `.pot` files as given and generate `.po` files. These will be sent along to a language-specific translation maintainer, who then compiles them to send to the R Core developer responsible for translations, who finally applies them as a patch.

To emphasize, this section is almost always not needed for contributing translations – it is here for completeness and edification.

10.1.2 `.po` files

.po files are the most important artifacts for translators. They provide the (human-readable!) mapping between the messages as they appear in the source code and how the messages will appear to users in translated locales.

10.1.2.1 Singular messages

Most messages appear as `msgid`/`msgstr` pairs. The former gives the message as it appears in the code, while the latter shows how it should appear in translation. For example, here is an error in German (locale: `de`) informing the user that their input must be of class `POSIXt`

```
msgid "'to' must be a \"POSIXt\" object"
msgstr "'to' muss ein \"POSIXt\" Objekt sein"
```

See this in context in the R-de.po source file.

The same message can also be found in R-it.po giving the translation to Italian:

```
msgid "'to' must be a \"POSIXt\" object"
msgstr "'to' dev'essere un oggetto \"POSIXt\""
```

10.1.2.2 Plural messages

Some messages will have different translations depending on some input determined at run time (e.g., the `length()` of an input object or the `nrow()` of a `data.frame`). This presents a challenge for translation, because different languages have different rules for how to pluralize different ordinal numbers[^See the relevant section of the `gettext` manual]. For example, English typically adds `s` to any quantity of items besides 1 (1 dog, 2 dog**s**, 100 dog**s**, even 0 dog**s**). Chinese typically does not alter the word itself in similar situations (, , ,); Arabic has *six* different ways to pluralize a quantity.

In `.po` files, this shows up in the form of `msgid_plural` entries, followed by several ordered `msgstr` entries. Here's an example from R-de.po[1]:

```
msgid "Warning message:\n"
msgid_plural "Warning messages:\n"
msgstr[0] "Warnmeldung:\n"
msgstr[1] "Warnmeldungen:\n"
```

The two entries in English correspond to the singular and plural messages; the two entries in German correspond similarly, because pluralization rules in German are similar to those in English. The situation in Lithuanian (R-lt.po) is more divergent:

```
msgid "Warning message:\n"
msgid_plural "Warning messages:\n"
msgstr[0] "Įspėjantis pranešimas:\n"
msgstr[1] "Įspėjantys pranešimai:\n"
msgstr[2] "Įspėjančių pranešimų:\n"
```

This corresponds to the 3 different ways to pluralize words in Polish.

What do `0`, `1`, and `2` correspond to, exactly? Ideally, this will be clear to native speakers of the language, but for clarity, it is the solution to a small arithmetic problem that can be found in the language's metadata entry. Look for the `Plural-Forms` entry in the metadata at the top of the `.po` file; here it is for Lithuanian:

[1]The GitHub mirror of the actual svn repo is linked in this chapter as it is a better interface for browsing the source files.

```
"Plural-Forms: nplurals=3; plural=(n%10==1 && n%100!=11 ? 0 : n%10>=2 && (n"
"%100<10 || n%100>=20) ? 1 : 2);\n"
```

`nplurals` tells us how many entries correspond to each `msgid_plural` for this language. `plural` tells us, for the quantity `n`, which entry to use. The arithmetic is C code; most important if you really want to parse this and are only familiar with R code is C's ternary operator: `test ? valueIfTrue : valueIfFalse` is a handy way to write R's `if (test) valueIfTrue else valueIfFalse`.

Parsing, we get the following associations:

- the `0` entry corresponds to when a number equals 1 modulo 10 (i.e., 1, 11, 21, 31, …) *except* numbers equaling 11 modulo 100 (i.e., 11, 111, 211, 311, …). Combining, that's 1, 21, 31, …, 91, 101, 121, 131, …, 191, …
- the `1` entry corresponds to numbers at least 2 modulo 10 (2, 3, …, 8, 9, 12, 13, 14, …) and *either* below 10 modulo 100 (0, 1, …, 9, 100, 101, …, 109, …) *or* exceeding 20 modulo 100 (21, 22, …, 99). Combining, that's 2, 3, …, 9, 22, 23, …, 29, 32, 33, … 39, …, 102, 103, …, 109, 122, 123, …
- The `2` entry corresponds to all other numbers, i.e. 0, 10, 11, 12, …, 19, 20, 30, …, 90, 100, 110, 111, 112, …

10.1.3 `.mo` files

`.po` files are plain text, but while helpful for human readers, this is inefficient for consumption by computers. The .mo format is a "compiled" version of the .po file optimized for retrieving messages when R is running.

In R-devel, the conversion from .po to .mo is done by R Core – you don't need to compile these files yourself. They are stored in the R sources at `./src/library/translations/inst` in various language-specific subdirectories.

10.2 How to contribute new translations

Translating R into different languages helps make it more user-friendly for non-English speakers and helps to grow the R community. See the blog post on how R can use your help: Translating R messages.

To get started contributing to translations, please follow the steps below:

Step 1: Register an Account at Weblate

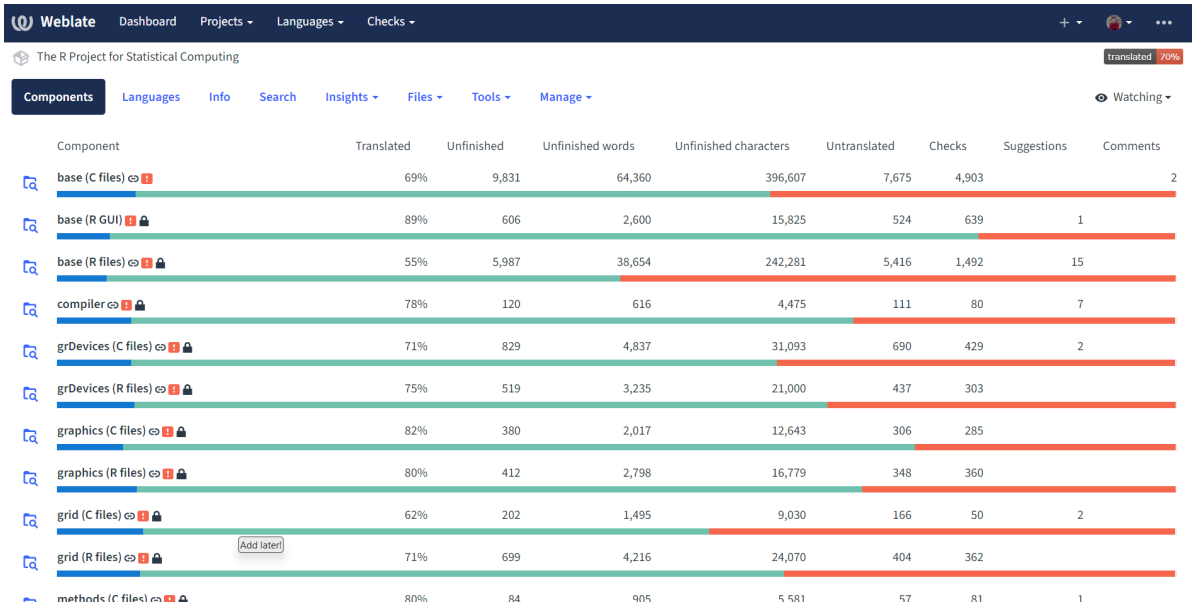
Weblate is an open-source platform for collaborative translation of software projects. Register an account on R's Weblate server, https://translate.rx.studio/, to start contributing.

Note: To get started follow the detailed workflow and convention for translation which is given here: https://contributor.r-project.org/translations/

Step 2: Choose a Component and Language

Select a component of R with less than 100% translation. Each component corresponds to the messages in either the R code or the C code of one of the packages in base R, e.g. base (R files), or tools (C files). There is one special case: base (R GUI), which corresponds to the messages in the Windows Graphical User Interface.

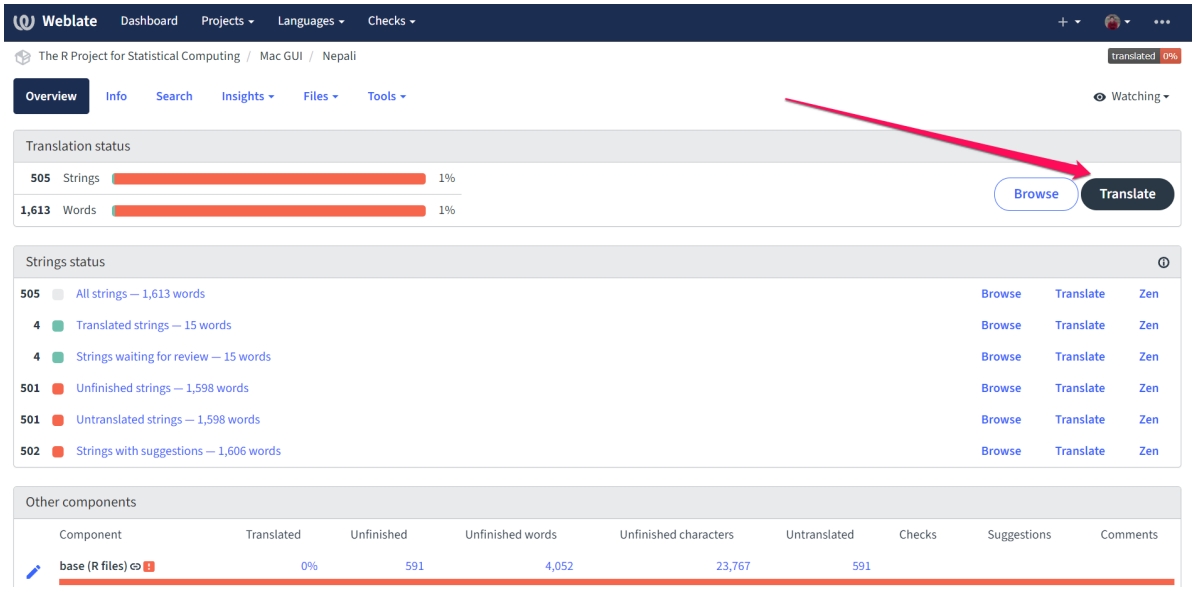
After selecting a component, you can select your preferred language.



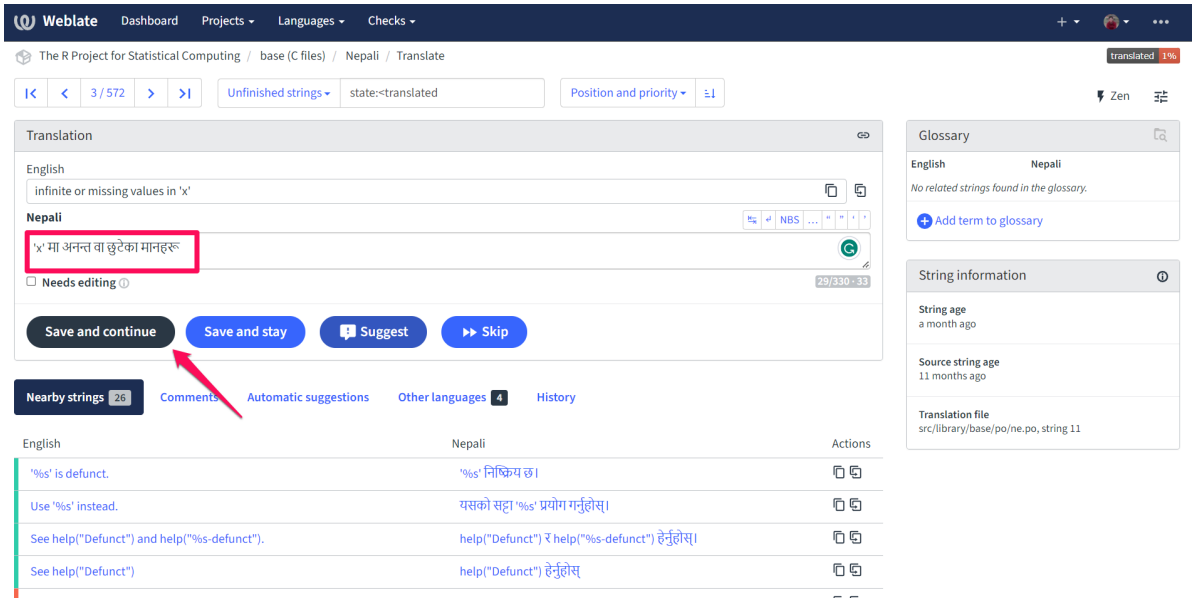
Step 3: Translate the Message

Now, you can click on Translate button on your right.

Note: More information for String status visit: https://docs.weblate.org/en/latest/workflows.html#translation-states



Then, start translating the message by typing the translation in the text box.

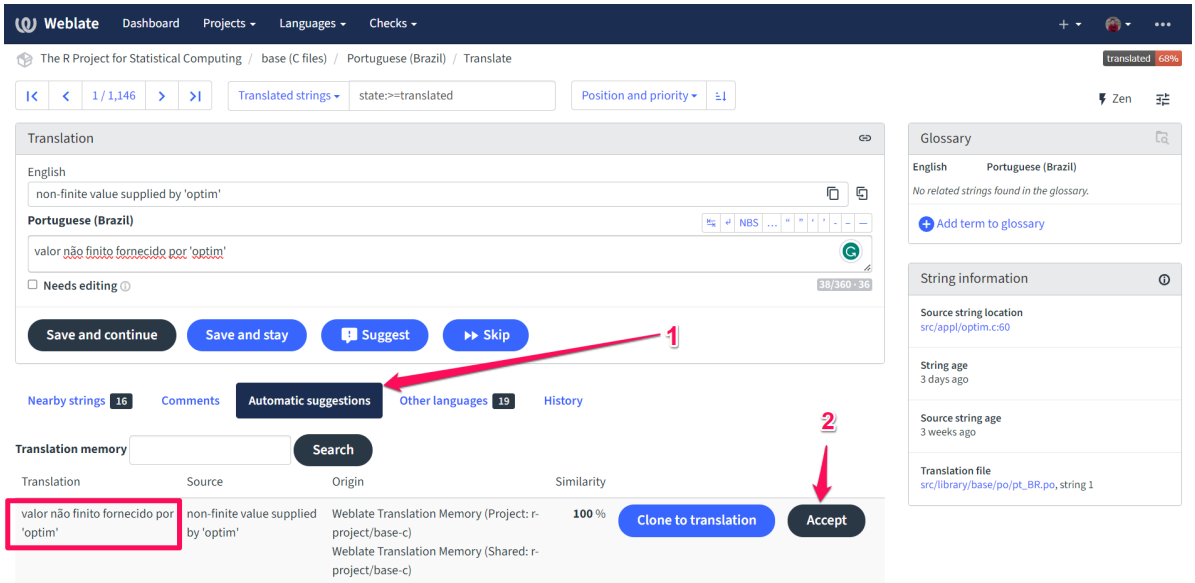


- If you are **confident** that the **translation is correct**, make sure the "Needs editing" box is **unchecked**.
- If you are **unsure** about how to translate, write the translation as a **Suggest** button instead.
- Finally, Click **"Save and Continue"** to save the translation and continue.

Note: Use Glossary feature within Weblate making translation easy and consistent: https://translate.rx.studio/projects/r-project/glossary/

Note: Make sure to use Automatic Suggestions as a starting point.

1. Click on Automatic Suggestions (machine translation)
2. Accept it if you think the automatic suggestion looks good



Some Tips to follow:

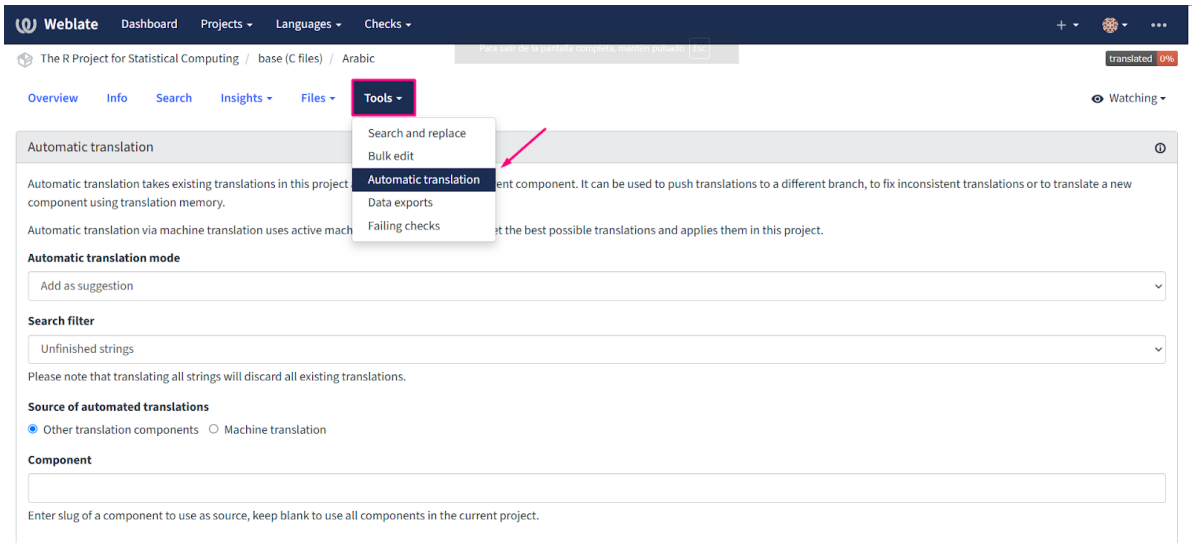
- **Be consistent:** Use the same words and phrases throughout the translation to make it consistent and avoid confusion.
- **Check for technical issues:** After finishing the translation, check if you have any alerts or warning in the Weblate string status, e.g. double instead of single space.
- **Follow language specific guidelines:** Check how other languages have translated the string. Even if you are not fluent in another language it can give you an idea of how other translators have handled it, especially which parts are left verbatim. A detailed guide is given here : Conventions-for-translations#languages-and-contributions

Related links: https://contributor.r-project.org/tutorials/translating-r-to-your-language/

10.3 Bulk Translations

Instead of translating one string at a time, it is possible to bulk translate a whole component of R at once:

1. Navigate to the translations for a particular component in a particular language, e.g. https://translate.rx.studio/projects/r-project/base-c/ar/
2. Select **'Tools > Automatic translation'**

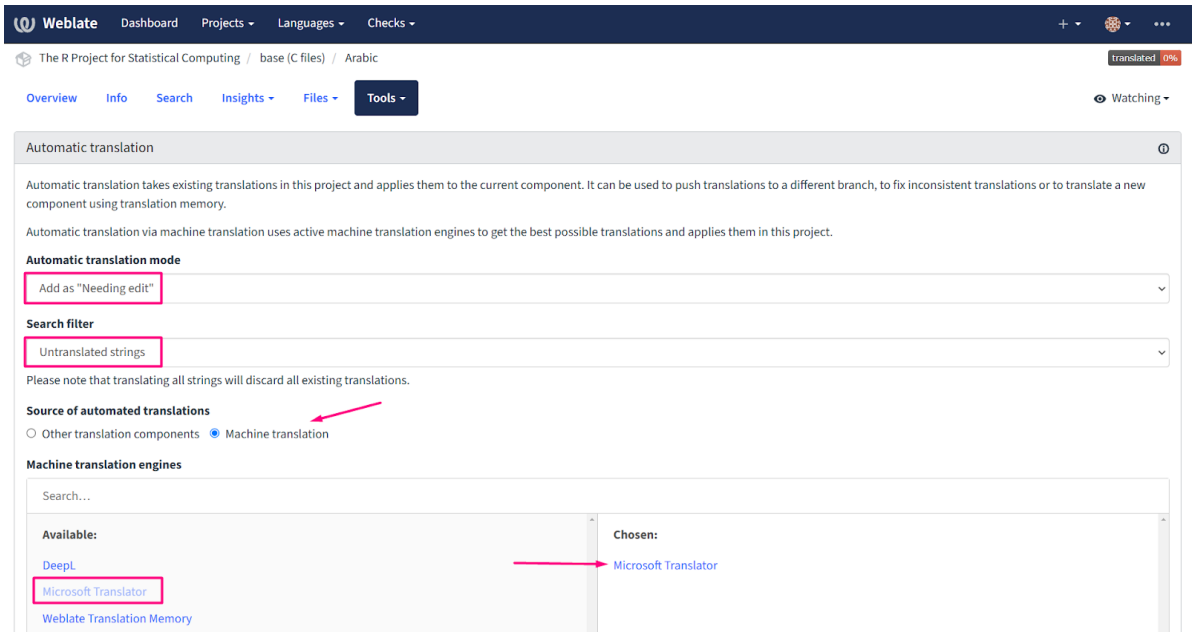


3. In the dialog,

- under **'Automatic translation mode'**, select **'Add as "needing edit"'**[2]:
- under **'Search filter'**, select **'Untranslated strings'**
- under **'Source of automated translations'**, select **'Machine translation'**
- under **'Machine translation engines'**, select **'Microsoft Translator'**[3]:

[2]By selecting **'Add as "needing edit"'**, the autotranslated strings would be treated as "fuzzy" translations, i.e. they would be added to the component source files when a patch is made, but they would NOT be used.

[3]Microsoft Translator is preferred for bulk translation as our free tier covers 2 million characters/month, where DeepL only allows 500k chars/month. If Microsoft Translator translations are much worse than DeepL, it may be useful to use it to translate one string at a time.



4. Review the translations, edit as necessary and uncheck **"needing edit"**.

10.4 Current status of translations in R

https://contributor.r-project.org/translations-dashboard/

10.5 Helpful references

- Statistical terms glossary
- The R Translations site contains more details on translating R messages and getting involved in existing projects.

11 Testing Pre-release R Versions

This chapter is inspired from the blog on testing R before release and discusses how you can help with testing of pre-release versions of R.

11.1 Where to test?

Whenever possible use a fresh package library for testing, even better would be to use virtual machines for the testing. This would ensure that you do not damage your existing R installation.

11.1.1 Virtual machine

A free Windows 10 virtual machine is provided by Microsoft (with a 90-day limit) for building, testing, and checking R packages and R itself. Package maintainers who work on Linux and MacOS can use it to test their packages on Windows. Read the instructions on how to automatically set up the machine to check R packages. Tomas Kalibera describes the details of using virtual machine in the blog Virtual Windows machine for checking R packages.

11.2 What can you test?

You can test:

- Your own programs.
- Your own workflows.
- Your special ways of installing or setting up R.
- Things that interact with external libraries.
- Interactive R packages.

Details of performing testing on various operating systems:

- Windows

- macOS
- Linux
- Solaris

11.3 Writing tests for R

Writing tests for R is much like writing tests for your own code. Tests need to be thorough, fast, isolated, consistently repeatable, and as simple as possible.

When you are adding tests to an existing test file, it is also recommended that you study the other tests in that file; it will teach you which precautions you have to take to make your tests robust and portable. We try to have tests both for normal behaviour and for error conditions. Tests live in the `tests` directory.

11.4 Benchmarks

Benchmarking is useful to test that a change does not degrade performance.

12 Where to Get Help

If you are working on R it is possible that you will come across an issue where you would need some assistance to solve it. If you require help, there are options available to seek assistance or get some feedback which are discussed in this chapter. If the question involves process or tool usage then please check the rest of this guide first as it should answer your question. Please make sure to search the documentation and resources to see if your question has already been addressed. If not, then ask for assistance in the appropriate forum. Many developers are volunteers and please be polite, patient, and thoughtful when requesting for feedback or help.

12.1 Slack

You can discuss issues related to the development of R and learn about the process of contributing to R on the R Contributors slack. There are a number of experienced developers on this slack who can answer questions and/or provide feedback. The following channels are available on the R-devel slack for help and feedback with specific areas:

- #bugreports-for-review: Share bug reports for review/feedback before submitting to Bugzilla.
- #core-dev-help: Getting help on anything related to R Core contribution.
- #core-documentation: Discuss patches/improvements to R's documentation.
- #core-translation: Discuss translating R messages, warnings, and errors into non-English languages.
- #patches-for-review: Share patches for peer review before submitting to R Core.

Note: You may not be able to access the history of these channels, so it cannot be used as a knowledge base of sorts.

12.2 Mailing lists

There are quite a few mailing lists for getting help with R:

- R-devel:
  - Questions and discussion about development *of* R vs. *with* R.
  - Getting help with technical programming issues, e.g. interfacing R with C/C++.
  - Proposals of new functionality/feature requests for R.
  - Pre-testing of new versions of R.
  - Enhancements and patches to the R source code and the R documentation.
  - Posting examples and benchmarks.
- R-help:
  - Discussions about problems and solutions using R.
- R-package-devel:
  - Getting help about package development in R.
  - Learning about the package development process.
  - Discussing problems developing a package (or problem in passing the R CMD check).

Please avoid cross-posting to both the R-package-devel and the R-devel mailing lists.

12.3 File a bug

If you strongly suspect you have come across a bug (be it in the build process, or in other areas), then report it on Bugzilla.

13 News and Announcements

Here are some resources that can be useful to keep up with the developments in R:

13.1 Blogs

The R project maintains The R Blog with posts mainly written by the R Core Team. News of changes in the development version of R found on the Daily News about R-devel blog which is updated daily.

13.2 Conferences

Updates about conferences actively supported or endorsed by The R Foundation can be found here. These conferences are organised by members from the R community.

13.3 Journal

The R Journal is an open access and refereed journal featuring short to medium length articles that should be of interest to users or developers of R. It also has a news section where information on, changes in R (new features of the latest release), changes on CRAN (new add-on packages, manuals, binary contributions, mirrors, etc.), upcoming conferences, and conference reports is provided.

13.4 Mailing lists

- R-announce: A moderated mailing list used for announcements by the R Core Development Team. Major announcements about the development of R and the availability of new code are made here.
- R-packages: A moderated mailing list for announcements about contributed R packages (typically on CRAN) and similar R project extensions.

13.5 Twitter

Follow @R_dev_news on Twitter for news of changes in the development version of R and new posts on The R Blog announcements.

14 Developer Tools

This chapter lists resources and tools which R developers may use. Here we will go over some commonly used tools that are relevant to R's workflow. As there are several ways to accomplish these tasks, this chapter reflects methods suitable for new contributors. Experienced contributors may desire a different approach.

14.1 Subversion (svn) client

Subversion (svn) is a version control system that tracks any changes made to files and directories. You can install either the TortoiseSVN (command line tool, and Windows Explorer integration) or the SlikSVN (just the command line tool) client. They have Windows installers and can be used from Windows cmd or RStudio terminal.

Some resources for learning subversion commands:

1. Apache Subversion quick start guide
2. TortoiseSVN commands
3. SlikSVN basics
4. Subversion book

14.2 Globally search for a regular expression and print matching lines (grep)

grep is a command line utility for searching plain text data sets for lines that match a regular expression. Refer the grep manual for more commands.

14.3 Git

Git is also a version control system for tracking changes in any files and directories. View git documentation for learning git commands.

14.4 GitHub

Some resources that are useful while using GitHub are:

1. Creating a pull request
2. Opening an issue from code
3. Resolving a merge conflict on GitHub

15 Additional resources

There are a number of further resources to help with contributions to R.

15.1 R Contributor site

Contains information for people interested in contributing to R core, including details for the R Contribution Working Group, the R Contributors Slack workspace and upcoming R Contributor events and meetups.

15.2 R developer page

The R developer page contains information about the plans of the R Core Team and details on the R release process, as well as pointers to technical write-ups including the R blog.

15.3 R manuals

There are a number of manuals created and included with R installations. The original latest and development versions of these are available from the R Manuals site. Note that the versions included on the site are created with Debian Linux. While there may be some variation for Windows and Mac manuals, most parts will be identical regardless of platform.

Particular manuals that might be of interest to R contributors are:

- R installation and administration - gives more details on building R from source.
- R internals - for those who want to read deeper into the internals R, for example, it contains chapters on special data types used in the C code, internal and primitive functions and details of graphics devices.
- R language definition - an in depth overview of the R language, helpful for developing more advanced R knowledge from a user's perspective.

There is a new re-styled R manuals site built with Quarto that has been produced to make searching and navigating the manuals easier in a web browser. This site is automatically built from the original manuals site using a number of scripts in the r-manuals GitHub repo.

16 R Core Developers

This page lists the current and former members of the R Core Team who have write access to the R source.

Current members of the R Core Team:

- John Chambers
- Peter Dalgaard
- Robert Gentleman
- Kurt Hornik
- Ross Ihaka
- Mikael Jagan
- Michael Lawrence
- Uwe Ligges
- Thomas Lumley
- Martin Maechler
- Sebastian Meyer
- Paul Murrell
- Martyn Plummer
- Brian Ripley
- Deepayan Sarkar
- Duncan Temple Lang
- Luke Tierney
- Heather Turner
- Simon Urbanek

Former members of the R Core team: * Tomas Kalibera (up to March 2026) * Friedrich Leisch (up to April 2024) * Douglas Bates (up to March 2024) * Martin Morgan (up to June 2021) * Duncan Murdoch (up to September 2017) * Seth Falcon (up to August 2015) * Stefano Iacus (up to July 2014) * Guido Masarotto (up to June 2003) * Heiner Schwarte (up to October 1999)

The Contributors page on the R Project website also lists specific contributors outside the R Core Team who have provided invaluable help by donating code, bug fixes, and documentation.